Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Sieci neuronowe

Klasyfikacja i regresja z użyciem perceptronu wielowarstwowego

Raport

Zespół:

Adam Grącikowski

Prowadzący:

dr inż. Maksymilian Bujok

mgr inż. Grzegorz Zakrzewski

Oświadczam, że niniejsza praca, stanowiąca podstawę do uznania osiągnięcia efektów uczenia się z przedmiotu *Sieci neuronowe*, została wykonana samodzielnie.

Warszawa, 18 października 2026

Streszczenie

W projekcie zaimplementowano od podstaw, bez bibliotek sieci neuronowych, perceptron wielowarstwowy uczony propagacją wsteczną błędu i zbadano go w zadaniach regresji funkcji jednej zmiennej i klasyfikacji punktów płaszczyzny.

Na czterech zbiorach danych, w czterech licznościach zbioru uczącego, zbadano wpływ funkcji aktywacji, architektury sieci i funkcji straty. Każdy wariant uczono dla 10 ziaren generatora liczb losowych, łącznie w 2400 eksperymentach, a wyniki porównywano na podstawie statystyk po ziarnach.

W klasyfikacji dla klas rozdzielonych prostą wystarczyła sieć bez warstw ukrytych, a dla nakładających się skupisk wystarczała jedna mała warstwa ukryta. Duże sieci uczone na małych zbiorach przeuczały się, a ich dokładność na zbiorze uczącym przewyższała testową nawet o 8,19 punktu procentowego. Funkcje aktywacji i funkcje straty dawały w klasyfikacji różnice rzędu dziesiątych części punktu procentowego.

W regresji błąd poza zakresem danych uczących zależał od kształtu przedłużenia funkcji sieci: dla funkcji o kształcie nasycającym sieć z funkcją tanh miała medianę MSE od 0,600 do 2,33, a sieci z ReLU i leaky ReLU od 1410 do 2200.

Wybór najlepszego wariantu na podstawie jednego ziarna różnił się od wyboru na podstawie 10 ziaren w 38% przypadków w klasyfikacji i 39% w regresji, co potwierdza potrzebę uczenia każdego wariantu wielokrotnie.

Spis treści

1 Wstęp **3**
1.1 Cel i zakres projektu . . . 3
1.2 Struktura raportu . . . 3

2 Opis problemu **4**
2.1 Regresja i klasyfikacja . . . 4
2.2 Zbiory danych . . . 4
2.3 Zakres badań . . . 5

3 Perceptron wielowarstwowy **7**
3.1 Model sieci . . . 7
3.2 Funkcje aktywacji . . . 7
3.3 Funkcje straty . . . 8
3.4 Propagacja wsteczna . . . 8
3.5 Optymalizacja . . . 9
3.6 Inicjalizacja wag . . . 10

4 Implementacja **11**
4.1 Architektura rozwiązania . . . 11
4.2 Konfiguracja eksperymentu . . . 11
4.3 Przygotowanie danych . . . 13
4.4 Sieć i uczenie . . . 13
4.5 Powtarzalność . . . 14
4.6 Weryfikacja poprawności . . . 14
4.7 Artefakty, wizualizacja i automatyzacja badań . . . 14

5 Wizualizacja procesu uczenia **15**
5.1 Klasyfikacja . . . 15
5.2 Regresja . . . 19

6 Metodyka eksperymentów **23**
6.1 Plan badań . . . 23
6.2 Konfiguracja bazowa . . . 23
6.3 Miary jakości i statystyki . . . 24
6.4 Założenia i ograniczenia planu . . . 25

7 Wyniki klasyfikacji **26**
7.1 Funkcje aktywacji . . . 26
7.2 Architektura sieci . . . 26
7.3 Funkcje straty . . . 29

8 Wyniki regresji **31**
8.1 Funkcje aktywacji . . . 31
8.2 Architektura sieci . . . 32
8.3 Funkcje straty . . . 35

9 Analiza przekrojowa **37**
9.1 Rozmiar zbioru uczącego i przeuczenie . . . 37
9.2 Powtarzalność między ziarnami . . . 38

10 Podsumowanie i wnioski **40**
10.1 Wnioski . . . 40
10.2 Ograniczenia i dalsze prace . . . 41

Bibliografia **42**

A Odtworzenie wyników **43**

1 Wstęp

W rozdziale przedstawiono cel i zakres projektu oraz strukturę raportu.

1.1 Cel i zakres projektu

Perceptron wielowarstwowy (ang. *multilayer perceptron*, MLP) jest siecią neuronową złożoną z kolejnych warstw neuronów, w której każda warstwa przekształca wyjście poprzedniej przez funkcję liniową i nieliniową funkcję aktywacji [1]. Parametry sieci dobiera się metodą spadku wzdłuż gradientu, a gradient funkcji straty względem wszystkich parametrów oblicza się algorytmem propagacji wstecznej błędu [2].

Celem projektu była implementacja perceptronu wielowarstwowego od podstaw, bez bibliotek implementujących sieci neuronowe, i zbadanie jego działania w dwóch zadaniach: regresji funkcji $f\colon \mathbb{R} \to \mathbb{R}$ oraz klasyfikacji punktów płaszczyzny $\mathbb{R}^2$. Na czterech dostarczonych zbiorach danych zbadano wpływ funkcji aktywacji, liczby warstw ukrytych i neuronów w nich oraz funkcji straty na jakość sieci. Każdy wariant uczono dla wielu ziaren generatora liczb losowych, ponieważ wynik pojedynczego uczenia zależy od losowej inicjalizacji wag i kolejności próbek. Raport dotyczy części pierwszej projektu. Część druga, rozpoznawanie cyfr ze zbioru MNIST, nie jest w nim opisana.

1.2 Struktura raportu

W rozdziale 2 opisano oba zadania, miary jakości, dostarczone zbiory danych i wymagany zakres badań. W rozdziale 3 przedstawiono model sieci, funkcje aktywacji i straty, propagację wsteczną, metody optymalizacji i inicjalizacji wag, a w rozdziale 4 ich implementację. Rozdział 5 zawiera wymagane wizualizacje procesu uczenia na przykładzie pojedynczych eksperymentów. W rozdziale 6 opisano plan badań, konfigurację bazową i sposób porównywania wariantów. Wyniki badań klasyfikacji i regresji przedstawiono w rozdziałach 7 i 8, a zależności wspólne dla wszystkich badań w rozdziale 9. Rozdział 10 podsumowuje wnioski i ograniczenia badań. W dodatku A podano polecenia potrzebne do odtworzenia wyników.

2 Opis problemu

W rozdziale opisano dwa rozwiązywane zadania, sposób oceny ich rozwiązań, dostarczone zbiory danych oraz zakres badań wymagany w projekcie.

2.1 Regresja i klasyfikacja

Oba zadania należą do uczenia nadzorowanego (ang. *supervised learning*). Dany jest zbiór uczący $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ par wejście-odpowiedź, na podstawie którego dobierane są parametry modelu. Celem nie jest jednak odtworzenie odpowiedzi ze zbioru uczącego, lecz poprawne przewidywanie dla nowych wejść. Dlatego jakość modelu ocenia się na osobnym zbiorze testowym, który nie jest używany w trakcie uczenia [1].

W zadaniu regresji wejściem jest liczba rzeczywista $x \in \mathbb{R}$, a odpowiedzią wartość $y \in \mathbb{R}$ pewnej nieznanej funkcji $f\colon \mathbb{R} \to \mathbb{R}$. Model zwraca przybliżenie $\hat{y}_i$ wartości y_i. Miarą jakości jest błąd średniokwadratowy (ang. *mean squared error*, MSE) liczony na m próbkach zbioru testowego w oryginalnej skali odpowiedzi:

$$\mathrm{MSE} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2 \,. \tag{1}$$

W zadaniu klasyfikacji wejściem jest wektor $\mathbf{x} \in \mathbb{R}^2$, a odpowiedzią numer klasy $y \in \{1, \ldots, K\}$. Model zwraca rozkład prawdopodobieństwa na K klasach, a decyzją $\hat{y}_i$ jest klasa o największym prawdopodobieństwie. Miarą jakości jest dokładność (ang. *accuracy*), czyli odsetek poprawnie sklasyfikowanych próbek zbioru testowego:

$$\mathrm{ACC} = \frac{1}{m} \sum_{i=1}^{m} \mathbf{1}\left[\hat{y}_i = y_i\right], \tag{2}$$

gdzie $\mathbf{1}[\cdot]$ przyjmuje wartość 1, gdy warunek jest spełniony, i 0 w przeciwnym razie. Jeśli rozkłady klas nachodzą na siebie, w części przestrzeni wejść ta sama wartość $\mathbf{x}$ może należeć do różnych klas. Nawet optymalny klasyfikator, zwany klasyfikatorem Bayesa, myli się wtedy na części próbek, więc dokładność 100% jest nieosiągalna [3].

Miary (1) i (2) służą do oceny i porównywania modeli. Wielkość minimalizowana w trakcie uczenia, czyli funkcja straty, nie musi być z nimi tożsama. Funkcje straty opisano w podrozdziale 3.3.

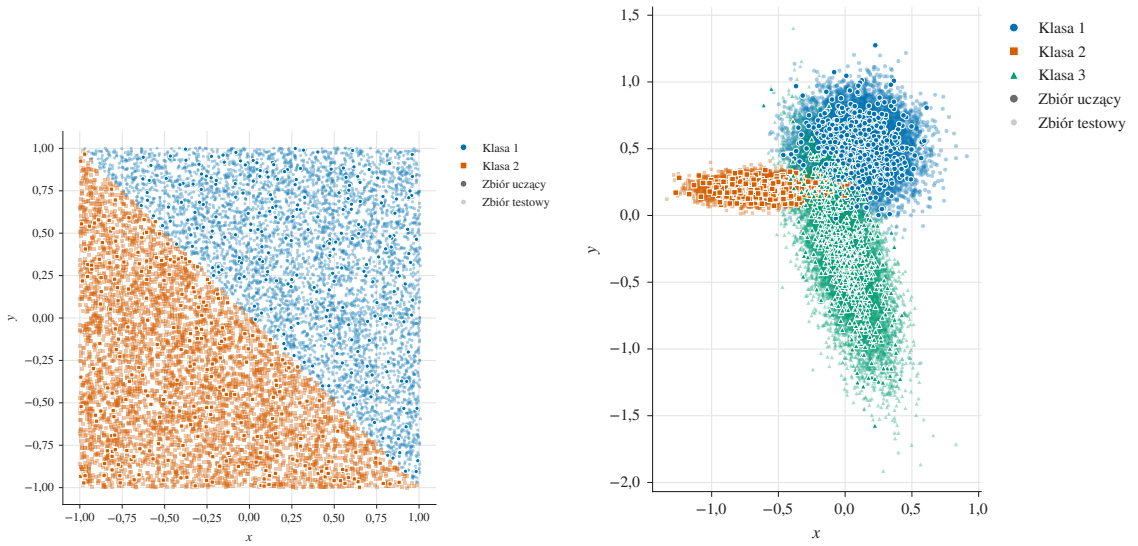
2.2 Zbiory danych

Do badań dostarczono cztery zbiory: dwa do klasyfikacji (`simple`, `three_gauss`) i dwa do regresji (`activation`, `cube`). Każdy występuje w czterech licznościach zbioru uczącego, a każdej liczności towarzyszy plik testowy. Liczności zestawiono w Tabeli 1.

Tabela 1: Dostarczone zbiory danych. Liczności podano jako liczbę próbek w pliku. W `three_gauss` każda klasa ma jedną trzecią próbek.

Zbiór	Zadanie	Liczba klas	Liczności zbioru uczącego	Zbiór testowy
`simple`	klasyfikacja	2	100, 500, 1000 i 10 000	tej samej liczności
`three_gauss`	klasyfikacja	3	300, 1500, 3000 i 30 000	tej samej liczności
`activation`	regresja	—	100, 500, 1000 i 10 000	1201 punktów
`cube`	regresja	—	100, 500, 1000 i 10 000	1201 punktów

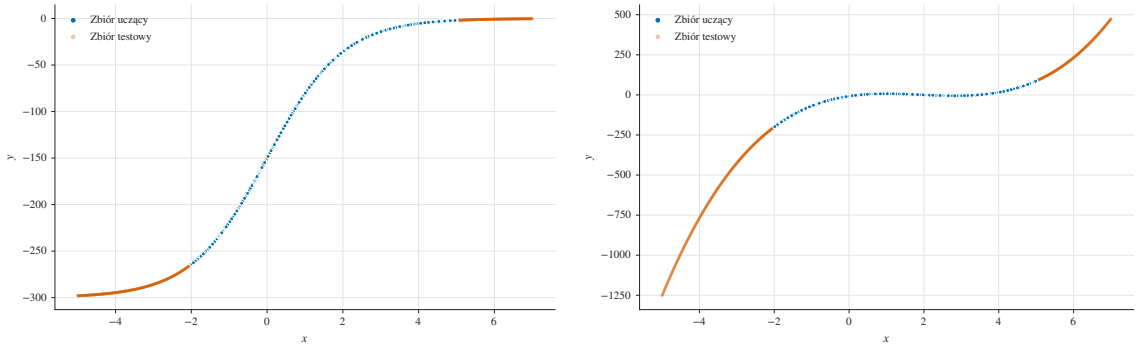
Zbiory klasyfikacji pokazano na Rysunku 1. W zbiorze `simple` punkty wypełniają kwadrat $[-1, 1]^2$, a dwie klasy są rozdzielone prostą biegnącą wzdłuż przekątnej. Pojedyncze punkty leżą po niewłaściwej stronie tej prostej. Zbiór `three_gauss` składa się z trzech równolicznych skupisk o różnych kształtach i rozrzutach, które częściowo na siebie nachodzą. W obszarze nakładania się skupisk przynależność punktu do klasy jest niejednoznaczna.



(a) `simple` (b) `three_gauss`

Rysunek 1: Zbiory klasyfikacji: zbiór uczący o liczności 500 na rysunku (a) i 1500 (500 na klasę) na rysunku (b) oraz zbiór testowy o liczności 10 000 na rysunku (a) i 30 000 na rysunku (b).

Zbiory regresji pokazano na Rysunku 2. Funkcja w zbiorze `activation` jest monotoniczna i nasyca się na obu krańcach dziedziny, przyjmując kształt litery S. Funkcja w zbiorze `cube` ma przebieg krzywej sześciennej: między dwoma łagodnymi zagięciami jest prawie płaska, a na krańcach szybko rośnie co do wartości bezwzględnej. W obu zbiorach odpowiedzi są rzędu setek, a punkty uczące leżą blisko krzywej wyznaczonej przez zbiór testowy.



(a) `activation` (b) `cube`

Rysunek 2: Zbiory regresji: zbiór uczący o liczności 500 i zbiór testowy o liczności 1201.

Zbiór testowy regresji jest jeden dla wszystkich liczności: 1201 punktów rozmieszczonych co 0,01 w przedziale $x \in [-5, 7]$. Wejścia zbiorów uczących leżą natomiast w przybliżeniu w przedziale $[-2, 5]$. Na części zbioru testowego model musi więc przewidywać wartości poza zakresem danych, na których był uczony, czyli ekstrapolować. Konsekwencje tego dla oceny modeli omówiono w podrozdziale 6.3 i rozdziale 8.

2.3 Zakres badań

Zadanie projektowe wymaga implementacji perceptronu wielowarstwowego uczonego algorytmem propagacji wstecznej błędu z metodą stochastycznego spadku wzdłuż gradientu (ang. *stochastic gradient descent*, SGD) lub jej wariantem, bez użycia bibliotek implementujących sieci neuronowe. Implementacja ma umożliwiać powtarzalne uczenie z zadanym ziarnem generatora liczb losowych oraz łatwą konfigurację liczby warstw, liczby neuronów w warstwach i obecności biasów. Wymagane są wizualizacje zbioru uczącego i wyników klasyfikacji oraz regresji, błędu w kolejnych iteracjach uczenia oraz wartości wag w kolejnych iteracjach.

Na dostarczonych zbiorach należy zbadać wpływ trzech czynników na skuteczność sieci:

- funkcji aktywacji w warstwach ukrytych: funkcji sigmoidalnej i co najmniej dwóch innych, przy aktywacji wyjściowej dobranej do rodzaju zadania,
- liczby warstw ukrytych, od 0 do 4, oraz liczby neuronów w nich,
- funkcji straty: dwóch dla klasyfikacji i dwóch dla regresji.

Wynik uczenia zależy od losowej inicjalizacji wag i kolejności podawania próbek, więc pojedyncze uczenie nie wystarcza do porównania wariantów. Zadanie wymaga zaadresowania tego problemu, dlatego każdy wariant uczono dla wielu ziaren generatora.

Model sieci i algorytm uczenia opisano w rozdziale 3, a ich implementację w rozdziale 4. Wymagane wizualizacje przedstawiono w rozdziale 5, a plan badań w rozdziale 6.

3 Perceptron wielowarstwowy

W rozdziale opisano model sieci, funkcje aktywacji i straty, algorytm propagacji wstecznej, metody optymalizacji oraz inicjalizację wag. Wzory odpowiadają implementacji opisanej w rozdziale 4. Opisano wszystkie zaimplementowane warianty i przy każdym zaznaczono, czy był używany w badaniach.

3.1 Model sieci

Perceptron wielowarstwowy jest złożeniem L warstw: $L-1$ warstw ukrytych i warstwy wyjściowej [1]. Implementacja dopuszcza od 0 do 4 warstw ukrytych. Kolejne warstwy mają $n_0 = d, n_1, \ldots, n_{L-1}, n_L = K$ neuronów, gdzie d to liczba cech wejściowych, a K to liczba wyjść: liczba klas w klasyfikacji i 1 w regresji. Sieć bez warstw ukrytych składa się z jednej warstwy o wymiarach $d \times K$.

Warstwy są reprezentowane jako macierze, a nie jako zbiory pojedynczych neuronów. Paczkę (ang. *batch*) n próbek zapisuje się jako macierz $X \in \mathbb{R}^{n \times d}$, w której wiersz odpowiada jednej próbce. Oznaczenia zestawiono w Tabeli 2. Propagacja w przód (ang. *forward pass*) przebiega od $A^{(0)} = X$ według wzoru

$$Z^{(l)} = A^{(l-1)} W^{(l)} + \mathbf{1}_n b^{(l)}, \qquad A^{(l)} = f_l\big(Z^{(l)}\big), \qquad l = 1, \ldots, L, \tag{3}$$

gdzie $\mathbf{1}_n$ jest kolumną jedynek, więc bias $b^{(l)}$ dodaje się do każdego wiersza. Funkcja aktywacji f_l działa na macierz $Z^{(l)}$ element po elemencie, z wyjątkiem funkcji softmax, która działa na całe wiersze. Wyjściem sieci jest $A^{(L)}$. Warstwa ma $n_{l-1} n_l$ wag i n_l biasów. Biasy można wyłączyć dla wszystkich warstw naraz, ale w badaniach były zawsze włączone.

Tabela 2: Oznaczenia używane w opisie sieci. Wiersz macierzy odpowiada jednej próbce paczki.

Symbol	Wymiar	Znaczenie
n, d, K, L	—	liczba próbek w paczce, cech wejściowych, wyjść i warstw
X	$n \times d$	wejścia paczki
Y	$n \times K$	cele: kodowanie *one-hot* klas albo wartości funkcji
$W^{(l)}$	$n_{l-1} \times n_l$	wagi warstwy l
$b^{(l)}$	$1 \times n_l$	biasy warstwy l
$Z^{(l)}$	$n \times n_l$	pobudzenia warstwy l (przed aktywacją)
$A^{(l)}$	$n \times n_l$	aktywacje warstwy l ($A^{(0)} = X$)
$\mathrm{d}M$	jak M	gradient straty $\partial \mathcal{L} / \partial M$ względem macierzy M

3.2 Funkcje aktywacji

W warstwach ukrytych dostępne są cztery funkcje aktywacji zestawione w Tabeli 3. Wszystkie były używane w badaniach. Funkcje ReLU (ang. *rectified linear unit*) i leaky ReLU nie są różniczkowalne w zerze. W implementacji przyjęto w tym punkcie pochodną lewostronną: 0 dla ReLU i α dla leaky ReLU [1].

Tabela 3: Funkcje aktywacji warstw ukrytych.

Funkcja	$f(z)$	$f'(z)$	Zbiór wartości
sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$\sigma(z)\big(1-\sigma(z)\big)$	$(0, 1)$
tanh	$\tanh z$	$1 - \tanh^2 z$	$(-1, 1)$
ReLU	$\max(0, z)$	1 dla $z > 0$, 0 dla $z \le 0$	$[0, \infty)$
leaky ReLU	z dla $z > 0$, αz dla $z \le 0$	1 dla $z > 0$, α dla $z \le 0$	$\mathbb{R}$

W badaniach przyjęto dla funkcji leaky ReLU $\alpha = 0{,}01$.

Aktywację wyjściową dobiera się do zadania. W regresji jest nią identyczność $f(z) = z$, więc sieć może zwrócić dowolną wartość rzeczywistą. W klasyfikacji używana jest funkcja softmax, która

przekształca wiersz pobudzeń $z \in \mathbb{R}^K$ w rozkład prawdopodobieństwa:

$$\operatorname{softmax}(z)_k = \frac{e^{z_k - m}}{\sum_{j=1}^{K} e^{z_j - m}}, \qquad m = \max_j z_j. \tag{4}$$

Odjęcie maksimum m nie zmienia wyniku, a chroni funkcję wykładniczą przed przepełnieniem. Zaimplementowano też funkcję sigmoidalną na wyjściu, której nie używano w badaniach.

Każde wyjście funkcji softmax zależy od wszystkich pobudzeń w wierszu, dlatego jej jakobian nie jest diagonalny: $\partial a_j / \partial z_i = a_j(\delta_{ij} - a_i)$ [3, rozdz. 4.3.4]. W propagacji wstecznej potrzebny jest jedynie iloczyn wektora gradientu przez jakobian, który dla całej paczki ma postać

$$\mathrm{d}Z = A \odot \left(\mathrm{d}A - (\mathrm{d}A \odot A)\, \mathbf{1}_K \mathbf{1}_K^\top\right), \tag{5}$$

gdzie $\odot$ oznacza mnożenie element po elemencie, a iloczyn $(\mathrm{d}A \odot A)\,\mathbf{1}_K$ to kolumna sum wierszy. Wzór (5) jest potrzebny dla straty MSE z wyjściem softmax.

3.3 Funkcje straty

Funkcja straty jest minimalizowana w trakcie uczenia. W odróżnieniu od miar jakości (1) i (2) liczy się ją na celach w postaci, w jakiej widzi je sieć: w klasyfikacji na kodowaniu *one-hot*, a w regresji na wartościach po normalizacji (podrozdział 4.3). Zaimplementowano trzy funkcje straty. Entropia krzyżowa (ang. *cross-entropy*) jest uśredniana po próbkach:

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{n} \sum_{s=1}^{n} \sum_{k=1}^{K} y_{sk} \ln \operatorname{softmax}\big(z_s\big)_k, \tag{6}$$

gdzie z_s to wiersz s macierzy $Z^{(L)}$, a softmax jest dany wzorem (4). Błąd średniokwadratowy i średni błąd bezwzględny (ang. *mean absolute error*, MAE) są uśredniane po wszystkich nK elementach macierzy wyjść:

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{nK} \sum_{s=1}^{n} \sum_{k=1}^{K} (a_{sk} - y_{sk})^2, \qquad \mathcal{L}_{\mathrm{MAE}} = \frac{1}{nK} \sum_{s=1}^{n} \sum_{k=1}^{K} |a_{sk} - y_{sk}|, \tag{7}$$

gdzie a_{sk} to elementy $A^{(L)}$.

Nie każda strata pasuje do każdej aktywacji wyjściowej. Dozwolone pary, sprawdzane przy wczytywaniu konfiguracji, zestawiono w Tabeli 4.

Tabela 4: Dozwolone pary funkcji straty i aktywacji wyjściowej.

Zadanie	Strata	Aktywacja wyjściowa	Użyta w badaniach
regresja	MSE	identyczność	tak
regresja	MAE	identyczność	tak
klasyfikacja	entropia krzyżowa	softmax	tak
klasyfikacja	MSE	softmax	tak
klasyfikacja	MSE	sigmoid	nie

3.4 Propagacja wsteczna

Gradient straty względem parametrów wszystkich warstw liczy się algorytmem propagacji wstecznej błędu (ang. *backpropagation*) [2]. Polega on na stosowaniu reguły łańcuchowej od warstwy wyjściowej do wejściowej i wykorzystaniu macierzy $Z^{(l)}$ i $A^{(l)}$ zapamiętanych w trakcie propagacji w przód [1].

Punktem wyjścia jest gradient względem pobudzeń warstwy wyjściowej. Dla entropii krzyżowej z funkcją softmax ma on prostą postać. Pochodna (6) po pobudzeniu z_{si} wynosi $\frac{1}{n}\sum_k y_{sk}(a_{si} - \delta_{ik}) = \frac{1}{n}(a_{si}\sum_k y_{sk} - y_{si})$, a ponieważ dla kodowania *one-hot* $\sum_k y_{sk} = 1$, otrzymuje się

$$\mathrm{d}Z^{(L)} = \frac{1}{n}\left(A^{(L)} - Y\right). \tag{8}$$

Dla pozostałych par najpierw liczy się gradient strat (7) względem wyjścia: $\mathrm{d}A^{(L)} = \frac{2}{nK}(A^{(L)} - Y)$ dla MSE oraz $\mathrm{d}A^{(L)} = \frac{1}{nK}\operatorname{sign}(A^{(L)} - Y)$ dla MAE, z przyjętym $\operatorname{sign}(0) = 0$. Następnie przekształca się go przez pochodną aktywacji wyjściowej, tak jak w warstwach ukrytych.

Dla kolejnych warstw, od $l = L$ do $l = 1$, gradienty wyznacza się ze wzorów

$$\begin{aligned} \mathrm{d}W^{(l)} &= A^{(l-1)\top}\,\mathrm{d}Z^{(l)}, \qquad \mathrm{d}b^{(l)} = \mathbf{1}_n^{\top}\,\mathrm{d}Z^{(l)}, \\ \mathrm{d}A^{(l-1)} &= \mathrm{d}Z^{(l)}\,W^{(l)\top}, \qquad \mathrm{d}Z^{(l-1)} = f'_{l-1}\big(Z^{(l-1)}\big) \odot \mathrm{d}A^{(l-1)}, \end{aligned} \tag{9}$$

przy czym dla funkcji softmax ostatni wzór zastępuje się wzorem (5). Suma po próbkach w $\mathrm{d}W^{(l)}$ i $\mathrm{d}b^{(l)}$ daje średnią po paczce, bo czynnik $1/n$ jest już zawarty w $\mathrm{d}Z^{(L)}$. Pełną procedurę przedstawiono w Algorytmie 1.

Algorytm 1 Gradient straty dla jednej paczki próbek

Wejście: paczka X, cele Y, parametry $W^{(l)}, b^{(l)}$ dla $l = 1, \ldots, L$
Wyjście: gradienty $\mathrm{d}W^{(l)}, \mathrm{d}b^{(l)}$ dla $l = 1, \ldots, L$

$A^{(0)} \leftarrow X$
for $l = 1, \ldots, L$ **do** ▷ propagacja w przód, wzór (3)
  $Z^{(l)} \leftarrow A^{(l-1)}W^{(l)} + \mathbf{1}_n b^{(l)}$
  $A^{(l)} \leftarrow f_l(Z^{(l)})$ ▷ $Z^{(l)}$ i $A^{(l)}$ są zapamiętywane
if strata to entropia krzyżowa, a wyjście to softmax **then**
  $\mathrm{d}Z^{(L)} \leftarrow (A^{(L)} - Y)/n$ ▷ wzór (8)
else
  $\mathrm{d}A^{(L)} \leftarrow \partial\mathcal{L}/\partial A^{(L)}$
  $\mathrm{d}Z^{(L)} \leftarrow f'_L(Z^{(L)}) \odot \mathrm{d}A^{(L)}$ ▷ dla funkcji softmax wzór (5)
for $l = L, \ldots, 1$ **do** ▷ wzory (9)
  $\mathrm{d}W^{(l)} \leftarrow A^{(l-1)\top}\mathrm{d}Z^{(l)}$
  $\mathrm{d}b^{(l)} \leftarrow \mathbf{1}_n^{\top}\mathrm{d}Z^{(l)}$
  if $l > 1$ **then**
    $\mathrm{d}A^{(l-1)} \leftarrow \mathrm{d}Z^{(l)}W^{(l)\top}$
    $\mathrm{d}Z^{(l-1)} \leftarrow f'_{l-1}(Z^{(l-1)}) \odot \mathrm{d}A^{(l-1)}$

3.5 Optymalizacja

Parametry sieci są aktualizowane metodą stochastycznego spadku wzdłuż gradientu z paczkami (ang. *mini-batch*). Gradient liczony na losowej paczce jest nieobciążonym estymatorem gradientu straty na całym zbiorze uczącym, a jego wyznaczenie kosztuje ułamek pełnego przejścia przez dane [1]. Zaimplementowano trzy reguły aktualizacji ze stałym współczynnikiem uczenia $\eta > 0$. Działają one element po elemencie, tak samo dla wag i biasów. Symbol θ oznacza dowolny parametr, a g jego gradient z bieżącej paczki. Podstawowa reguła SGD ma postać

$$\theta \leftarrow \theta - \eta\, g. \tag{10}$$

Metoda momentum [4] gromadzi w buforze v wykładniczo ważoną sumę poprzednich kroków, co tłumi oscylacje i przyspiesza ruch wzdłuż stałego kierunku spadku:

$$v \leftarrow \mu v - \eta\, g, \qquad \theta \leftarrow \theta + v, \qquad \mu \in [0, 1). \tag{11}$$

Metoda Adam [5] utrzymuje wykładnicze średnie pierwszego i drugiego momentu gradientu, koryguje ich obciążenie wynikające ze startu od zera i skaluje krok osobno dla każdego parametru:

$$\begin{aligned} t &\leftarrow t + 1, \qquad m \leftarrow \beta_1 m + (1-\beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1-\beta_2)\, g^2, \\ \hat{m} &= \frac{m}{1-\beta_1^{t}}, \qquad \hat{v} = \frac{v}{1-\beta_2^{t}}, \qquad \theta \leftarrow \theta - \eta\,\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}. \end{aligned} \tag{12}$$

Domyślne wartości to $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ i $\epsilon = 10^{-8}$. Bufory v, m i licznik t startują od zera. W badaniach używano wyłącznie metody Adam z domyślnymi wartościami β_1, β_2 i ϵ.

Przebieg uczenia przedstawiono w Algorytmie 2. Na początku każdej epoki kolejność próbek zbioru uczącego jest losowo permutowana, a następnie zbiór dzieli się na kolejne paczki o rozmiarze B. Ostatnia, niepełna paczka jest zachowywana, więc epoka liczy $\lceil N/B \rceil$ kroków, gdzie N to liczność zbioru uczącego. Wyznaczanie metryk, wykrywanie rozbieżności uczenia i wczesne zatrzymanie opisano w podrozdziale 4.4.

Algorytm 2 Uczenie sieci metodą mini-batch

Wejście: zbiór uczący (X, Y) o liczności N, liczba epok E, rozmiar paczki B, reguła aktualizacji (10), (11) lub (12)
Wyjście: nauczone parametry sieci

```
wylosuj wagi (podrozdział 3.6), wyzeruj biasy i bufory optymalizatora
for e = 1, ..., E do
    wylosuj permutację π indeksów 1, ..., N
    for j = 1, ..., ⌈N/B⌉ do
        I ← indeksy π od (j − 1)B + 1 do min(jB, N)
        oblicz gradienty dla paczki (X_I, Y_I) Algorytmem 1
        zaktualizuj wszystkie parametry wybraną regułą
```

3.6 Inicjalizacja wag

Początkowe wagi są losowe, żeby neurony jednej warstwy nie pozostawały identyczne. Ich skalę dobiera się tak, by wariancja sygnału w propagacji w przód i gradientu w propagacji wstecznej nie malała ani nie rosła z warstwy na warstwę. Zaimplementowano trzy schematy, w których $n_{\text{in}} = n_{l-1}$ i $n_{\text{out}} = n_l$ to wymiary macierzy $W^{(l)}$. Inicjalizacja Xaviera [6] losuje wagi z rozkładu jednostajnego:

$$W_{ij}^{(l)} \sim \mathcal{U}[-r, r], \qquad r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \tag{13}$$

co daje wariancję $2/(n_{\text{in}} + n_{\text{out}})$. Została wyprowadzona dla funkcji symetrycznych względem zera, takich jak tangens hiperboliczny. Inicjalizacja He [7] uwzględnia, że ReLU zeruje około połowy pobudzeń, i losuje wagi z rozkładu normalnego o dwukrotnie większej wariancji względem n_{in}:

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right). \tag{14}$$

Trzeci schemat losuje wagi z rozkładu $\mathcal{U}[-r, r]$ o zadanym r. Biasy zawsze startują od zera, a wybrany schemat stosuje się do wszystkich warstw. W badaniach inicjalizację Xaviera (13) łączono z funkcjami sigmoidalną i tangensem hiperbolicznym, a inicjalizację He (14) z ReLU i leaky ReLU.

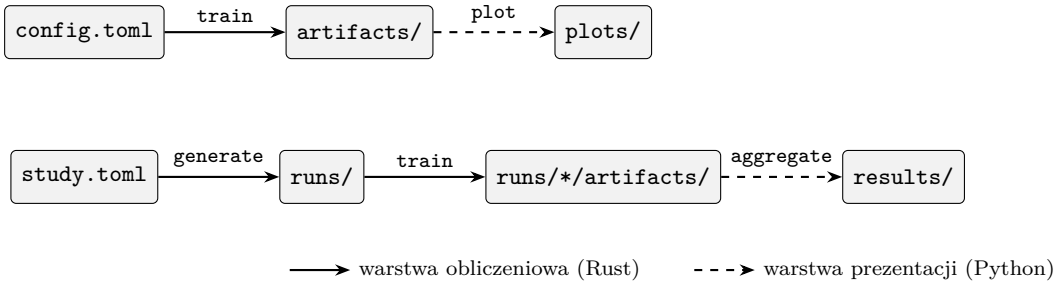
4 Implementacja

W rozdziale opisano budowę programu: podział na warstwy, konfigurację eksperymentu, przygotowanie danych, realizację uczenia, zapewnienie powtarzalności, weryfikację poprawności oraz zapis i analizę wyników.

4.1 Architektura rozwiązania

Program składa się z dwóch warstw, które komunikują się wyłącznie przez pliki na dysku. Warstwa obliczeniowa, napisana w języku Rust, wczytuje konfigurację i dane, uczy sieć i zapisuje surowe wyniki, nazywane dalej artefaktami. Warstwa prezentacji, napisana w języku Python, czyta artefakty i tworzy z nich wykresy oraz zestawienia.

Przepływ danych pokazano na Rysunku 3. Pojedynczy eksperyment jest opisany plikiem `config.toml`. Polecenie `compute train` zapisuje jego artefakty, a polecenie `plot` tworzy z nich wykresy. Badanie, czyli siatka eksperymentów różniących się wybranymi parametrami, jest opisane plikiem `study.toml`. Polecenie `compute generate` tworzy z niego katalogi pojedynczych eksperymentów, `compute train` uczy je równolegle, a `aggregate` zestawia wyniki.



Rysunek 3: Przepływ danych w programie. Górny wiersz dotyczy pojedynczego eksperymentu, dolny badania złożonego z wielu eksperymentów. Prostokąty oznaczają pliki i katalogi, a strzałki polecenia.

4.2 Konfiguracja eksperymentu

Wszystkie parametry eksperymentu, łącznie z ziarnem generatora liczb losowych, zapisuje się w jednym pliku `config.toml` w formacie TOML. Na Listingu 4.1 przedstawiono przykładowy plik eksperymentu pokazowego omawianego w rozdziale 5. Dostępne klucze i ich dozwolone wartości zestawiono w Tabeli 5. Liczbę warstw, liczbę neuronów w warstwach i obecność biasów zmienia się kluczami `model.hidden_layers` i `model.bias`. Przed uczeniem konfiguracja jest sprawdzana, m.in. pod kątem dozwolonych par z Tabeli 4, a komunikaty o błędach wskazują linię pliku.

Listing 4.1 Plik `config.toml` eksperymentu pokazowego `showcase-three-gauss`.

```toml
task = "classification"

[data]
train = "data/datasets/classification/data.three_gauss.train.500.csv"
test = "data/datasets/classification/data.three_gauss.test.10000.csv"  # common test set of the
    studies
target = "cls"
input_normalization = "standard"
target_normalization = "none"
validation_fraction = 0.0

[model]
hidden_layers = [16, 16]
bias = true
hidden_activation = { kind = "tanh" }
output_activation = "softmax"
init = { kind = "xavier" }

[optimizer]
kind = "adam"
learning_rate = 0.003

[training]
loss = "cross_entropy"
epochs = 300
batch_size = 32
seed = 1

[logging]
loss_every = 1
weights_every = 10                  # the studies use 100; denser snapshots give smooth trajectories
max_weights_per_layer = 32

[output]
grid_resolution = 100
grid_margin = 0.1
```

Tabela 5: Klucze pliku konfiguracyjnego. W nawiasach podano dodatkowe parametry wariantu, zapisywane obok jego nazwy w kluczu `kind`, np. `{ kind = "leaky_relu", alpha = 0.01 }`.

Klucz	Dozwolone wartości i znaczenie
`task`	`classification`, `regression`
`data.train`, `data.test`	ścieżki plików CSV o tych samych kolumnach
`data.target`	nazwa kolumny celu, pozostałe kolumny są cechami
`data.input_normalization`, `data.target_normalization`	`none`, `standard`, `min_max` (cel w klasyfikacji: `none`)
`data.train_subset`	opcjonalna liczność losowego podzbioru zbioru uczącego
`data.validation_fraction`	część zbioru uczącego wydzielana do walidacji, z przedziału $[0, 1)$
`model.hidden_layers`	liczności od 0 do 4 warstw ukrytych, np. `[16, 16]`
`model.bias`	`true`, `false`
`model.hidden_activation`	`sigmoid`, `tanh`, `relu`, `leaky_relu` (`alpha`)
`model.output_activation`	`identity`, `sigmoid`, `softmax`
`model.init`	`xavier`, `he`, `uniform` (`limit`)
`optimizer.kind`, `optimizer.learning_rate`	`sgd`, `momentum` (`momentum`), `adam` (`beta1`, `beta2`, `epsilon`) oraz współczynnik uczenia η
`training.loss`	`cross_entropy`, `mse`, `mae`
`training.epochs`, `training.batch_size`, `training.seed`	liczba epok, rozmiar paczki, ziarno generatora
`training.early_stopping`	opcjonalne wczesne zatrzymanie (`patience`, `min_delta`)
`logging.loss_every`, `logging.weights_every`	co ile iteracji zapisywać stratę i wagi
`logging.max_weights_per_layer`	największa liczba zapisywanych parametrów warstwy
`output.grid_resolution`, `output.grid_margin`	liczba punktów na oś i margines siatki predykcji dla jednej lub dwóch cech

4.3 Przygotowanie danych

Zbiory wczytuje się z plików CSV z nagłówkiem. Kolumnę celu wskazuje konfiguracja, a pozostałe kolumny są cechami. W klasyfikacji etykietami muszą być liczby całkowite. Klasami są różne etykiety zbioru uczącego posortowane rosnąco, więc etykiety $1, 2, 3$ otrzymują indeksy $0, 1, 2$. Cele są kodowane jako wektory *one-hot*, w których jedynka stoi na pozycji klasy, a pozostałe elementy są zerami [3, rozdz. 4].

Konfiguracja pozwala uczyć sieć na losowym podzbiorze zbioru uczącego i wydzielić z niego zbiór walidacyjny. W klasyfikacji oba losowania są stratyfikowane, tzn. zachowują proporcje klas.

Cechy, a w regresji także cel, mogą być normalizowane osobno dla każdej kolumny: standaryzacją $x' = (x - \mu)/s$ albo przekształceniem zakresu do przedziału $[0, 1]$. Parametry normalizacji wyznacza się wyłącznie na zbiorze uczącym i stosuje bez zmian do pozostałych zbiorów, więc informacja ze zbioru testowego nie trafia do uczenia. Sieć regresyjna przewiduje cel w skali znormalizowanej, dlatego błędy MSE i MAE raportuje się po odwróceniu normalizacji, w oryginalnej skali odpowiedzi, zgodnie ze wzorem (1). W badaniach standaryzowano wejścia oraz cel w regresji.

4.4 Sieć i uczenie

Warstwy są przechowywane jako macierze liczb zmiennoprzecinkowych podwójnej precyzji, w których wiersz odpowiada próbce, zgodnie z notacją z podrozdziału 3.1. Propagacja wsteczna realizuje Algorytm 1.

Pętla uczenia realizuje Algorytm 2. Przed uczeniem i po każdej epoce program wyznacza na zbiorach uczącym, walidacyjnym i testowym wartość straty oraz dokładność w klasyfikacji albo MSE i MAE w oryginalnej skali w regresji. Zbiór testowy jest tylko obserwowany i nie wpływa na przebieg uczenia. Jeśli strata przyjmie wartość nieokreśloną lub nieskończoną, uczenie kończy się ze statusem rozbieżności (ang. *diverged*), a krok optymalizatora z taką stratą nie jest wykonywany. Opcjonalne wczesne zatrzymanie przerywa uczenie, gdy strata walidacyjna nie poprawi się o więcej niż `min_delta` przez `patience` kolejnych epok, i przywraca sieć z najlepszej epoki. W badaniach wczesnego zatrzymania nie używano.

W trakcie uczenia program wypisuje konfigurację, liczbę parametrów sieci i w równych odstępach, domyślnie około 20 razy, wartości metryk. Przykład dla eksperymentu z Listingu 4.1 przedstawiono na Listingu 4.2.

Listing 4.2 Wyjście polecenia `compute train` dla eksperymentu `showcase-three-gauss` z wypisywaniem metryk co 60 epok.

```
config:     data/experiments/classification/showcase-three-gauss\config.toml
task:       classification
train:      data/datasets/classification/data.three_gauss.train.500.csv
test:       data/datasets/classification/data.three_gauss.test.10000.csv
columns:    2 feature(s), target `cls`
normalize:  inputs standard, target none
network:    2 -> 16 -> 16 -> classes
layers:     hidden tanh, output softmax, init xavier, bias on
loss:       cross_entropy
optimizer:  adam(lr=0.003, beta1=0.9, beta2=0.999, epsilon=1e-8)
training:   300 epoch(s), batch size 32, seed 1
samples:    train 1500, test 30000
classes:    1, 2, 3
  train      500 / 500 / 500
  test       10000 / 10000 / 10000
parameters: 371 (2x16+16, 16x16+16, 16x3+3)
epoch   0/300  train loss 1.2328 acc 36.27% | test loss 1.2409 acc 35.91%
epoch  60/300  train loss 0.1458 acc 94.40% | test loss 0.1683 acc 93.60%
epoch 120/300  train loss 0.1402 acc 94.60% | test loss 0.1652 acc 93.51%
epoch 180/300  train loss 0.1389 acc 94.60% | test loss 0.1693 acc 93.34%
epoch 240/300  train loss 0.1378 acc 94.60% | test loss 0.1675 acc 93.49%
epoch 300/300  train loss 0.1358 acc 94.67% | test loss 0.1708 acc 93.31%
stop:       completed after 300 epoch(s), 14100 iteration(s) in 4.74 s
result:     train loss 0.1358 acc 94.67% | test loss 0.1708 acc 93.31%
```

4.5 Powtarzalność

Przebieg uczenia jest w pełni wyznaczony przez konfigurację, w tym ziarno. Liczby losowe pochodzą z generatora ChaCha8 z biblioteki `rand` [8], opartego na szyfrze strumieniowym ChaCha z ośmioma rundami [9]. Z jednego ziarna tworzy się cztery niezależne strumienie tego generatora: dla losowań danych (podzbiór i walidacja), inicjalizacji wag, tasowania paczek oraz wyboru zapisywanych wag. Dzięki temu zmiana jednego elementu nie zmienia pozostałych losowań. Przykładem są eksperymenty pokazowe z rozdziału 5, które zapisują wagi dziesięć razy częściej niż eksperymenty badań, a mimo to dają wyniki identyczne co do bitu z odpowiadającymi im eksperymentami badań. Wynik nie zależy też od liczby równolegle uczonych sieci. Do artefaktów zapisywana jest kopia konfiguracji i wersja kodu.

4.6 Weryfikacja poprawności

Implementację propagacji wstecznej sprawdzono testami, które porównują gradient analityczny z numerycznym, liczonym różnicami centralnymi $(\mathcal{L}(\theta+h)-\mathcal{L}(\theta-h))/(2h)$. Testy obejmują wszystkie funkcje aktywacji warstw ukrytych, wszystkie pary z Tabeli 4 oraz sieci z różną liczbą warstw ukrytych, w tym bez warstw ukrytych i bez biasów. Osobne testy sprawdzają, że to samo ziarno daje ten sam przebieg uczenia.

4.7 Artefakty, wizualizacja i automatyzacja badań

Po uczeniu program zapisuje artefakty w katalogu `artifacts/` eksperymentu. Ich zawartość zestawiono w Tabeli 6. Tablice zapisuje się w binarnym formacie `.npy`, metryki epok w pliku CSV, a metadane w formacie JSON. Dla dużych warstw zapis wszystkich wag w każdej iteracji byłby zbyt kosztowny. Dlatego zapisywany jest stały, losowo wybrany podzbiór co najwyżej `max_weights_per_layer` parametrów warstwy, co `weights_every` iteracji. Normy wag liczy się zawsze ze wszystkich parametrów.

Tabela 6: Zawartość katalogu `artifacts/` eksperymentu.

Plik lub katalog	Zawartość
`metadata.json`	kopia konfiguracji, wersja kodu, opis danych i sieci, powód zatrzymania, liczba epok i iteracji, końcowe metryki, czas uczenia
`epochs.csv`	strata i metryki każdego zbioru przed uczeniem i po każdej epoce
`history/`	strata paczek uśredniona w oknach `loss_every` iteracji
`weights/`	wartości zapisywanych parametrów w kolejnych zapisach oraz normy $\|W\|_F$ i $\|b\|_2$ każdej warstwy
`model/`	końcowe wagi i biasy
`predictions/`	wyjścia sieci, cele lub etykiety oraz numery wierszy pliku CSV dla każdego zbioru
`grid/`	wyjścia sieci na siatce punktów przestrzeni wejść (dla jednej lub dwóch cech)

Warstwa prezentacji tworzy dla pojedynczego eksperymentu wykresy zbioru danych, granicy decyzyjnej albo krzywej regresji, straty w kolejnych iteracjach i epokach, metryk, trajektorii i norm wag, mapy wartości wag oraz macierze pomyłek.

Badanie opisuje plik `study.toml`, który zawiera konfigurację bazową bez ziarna, osie badanych parametrów z ich poziomami i listę ziaren. Polecenie `generate` tworzy po jednym eksperymencie dla każdej kombinacji poziomów i każdego ziarna oraz sprawdza każdą wygenerowaną konfigurację. Polecenie `aggregate` zestawia wyniki wszystkich eksperymentów badania w tabelach (wynik każdego eksperymentu oraz statystyki po ziarnach dla każdego wariantu), wykresach porównawczych i podsumowaniu. Skrypt `run-studies.ps1` wykonuje te kroki kolejno dla wszystkich badań i po przerwaniu wznawia pracę od eksperymentów jeszcze nienauczonych. Plan badań opisano w rozdziale 6.

5 Wizualizacja procesu uczenia

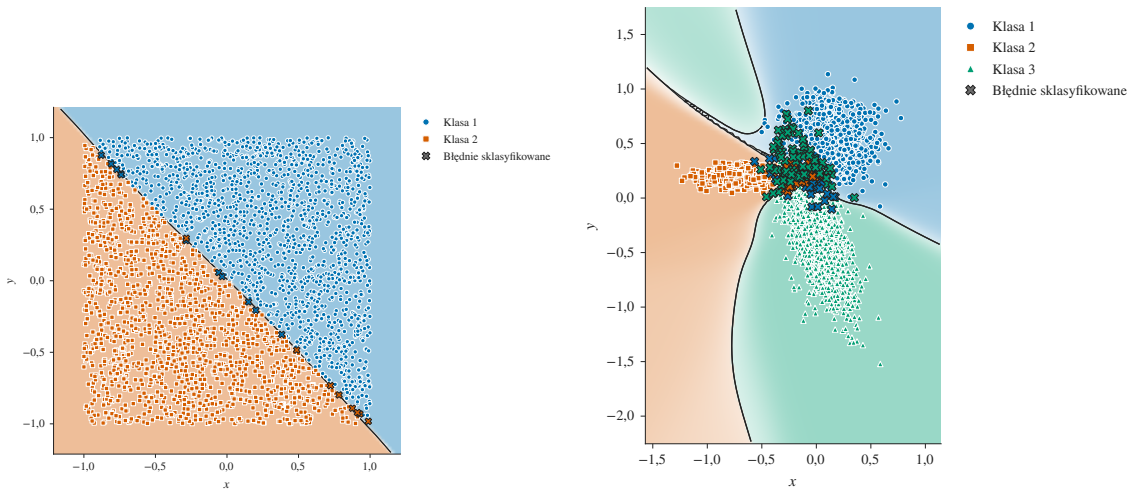
W rozdziale przedstawiono wymagane wizualizacje: wynik klasyfikacji i regresji, przebieg błędu w kolejnych iteracjach oraz ewolucję wartości wag. Pokazano je na czterech eksperymentach pokazowych, po jednym dla każdego zbioru. Każdy z nich jest jednym z eksperymentów badań opisanych w rozdziale 6, z wagami zapisywanymi co 10 zamiast co 100 iteracji (podrozdział 4.5). Wszystkie cztery sieci mają dwie warstwy ukryte z funkcją tanh, inicjalizację Xaviera i standaryzowane wejścia. Uczono je przez 300 epok metodą Adam ze współczynnikiem uczenia $\eta = 0{,}003$ i paczkami po 32 próbki, z ziarnem 1. Architektury i wyniki zestawiono w Tabeli 7. Pojedyncze uczenie ilustruje działanie sieci, natomiast porównania wariantów na podstawie wielu ziaren przedstawiono w rozdziałach 7–9.

Tabela 7: Eksperymenty pokazowe. Architekturę podano jako liczby neuronów kolejnych warstw, od wejścia do wyjścia. Metryka końcowa to dokładność w klasyfikacji i MSE w oryginalnej skali w regresji.

Zbiór	Architektura	Strata	Parametry	Iteracje	Metryka końcowa	
					uczący	testowy
`simple`	2-16-16-2	entropia krzyżowa	354	4800	99,80%	99,38%
`three_gauss`	2-16-16-3	entropia krzyżowa	371	14 100	94,67%	93,31%
`activation`	1-32-32-1	MSE	1153	4800	0,0156	9,21
`cube`	1-32-32-1	MSE	1153	4800	0,693	28 991

5.1 Klasyfikacja

Wyniki klasyfikacji pokazano na Rysunku 4. Kolor tła oznacza klasę wybraną przez sieć w danym punkcie płaszczyzny, a jego nasycenie pewność tej decyzji, czyli największe prawdopodobieństwo zwrócone przez funkcję softmax. Czarne linie to granice decyzyjne. Na tło naniesiono próbkę punktów zbioru testowego, a punkty sklasyfikowane błędnie oznaczono krzyżykami. W zbiorze `simple` sieć wyznaczyła granicę bliską prostej i myli się na 62 z 10 000 punktów testowych. Błędnie sklasyfikowane punkty widoczne na rysunku leżą wzdłuż granicy. W zbiorze `three_gauss` granice są zakrzywione, a błędy skupiają się w obszarze, w którym klasy nachodzą na siebie i pewność sieci jest najmniejsza.

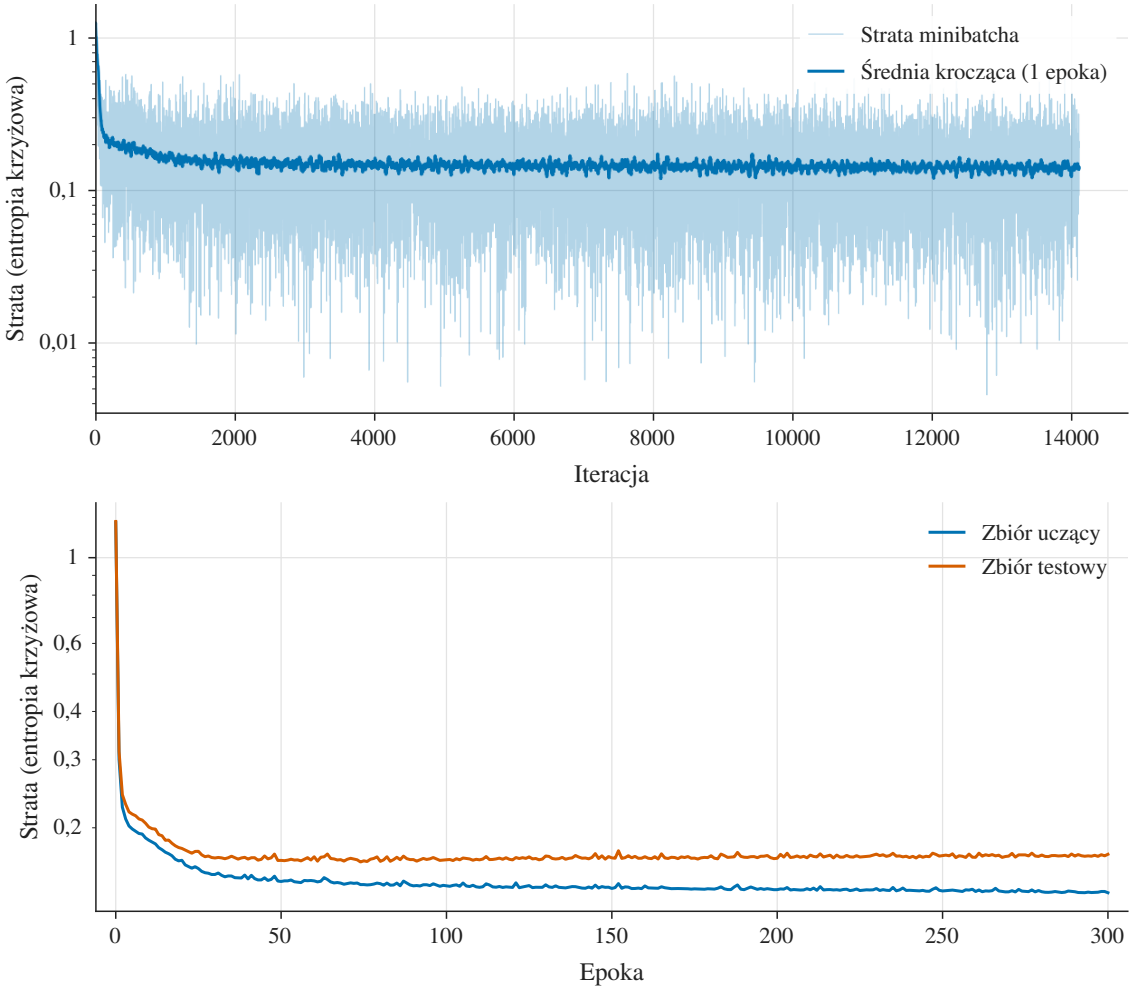


(a) `simple` (b) `three_gauss`

Rysunek 4: Obszary decyzyjne sieci i 3000 równomiernie wybranych punktów zbioru testowego. Krzyżyki oznaczają punkty sklasyfikowane błędnie.

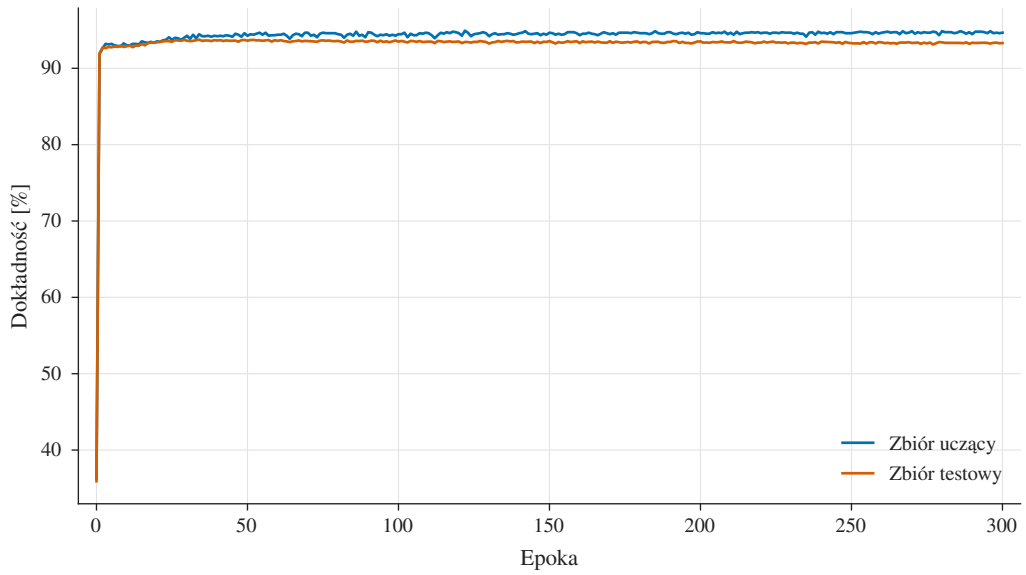
Przebieg błędu w kolejnych iteracjach uczenia na zbiorze `three_gauss` przedstawiono na Rysunku 5. Górny wykres pokazuje stratę każdej paczki i jej średnią kroczącą z jednej epoki. Zbiór uczący liczy 1500 próbek, więc epoka składa się z $\lceil 1500/32 \rceil = 47$ paczek, czyli iteracji. Strata pojedynczej paczki jest silnie zaszumiona, bo paczka 32 próbek trafia raz w obszar łatwy, a raz w obszar nakładania się klas. Dolny wykres pokazuje stratę na całych zbiorach po każdej epoce.

Największy spadek następuje w pierwszej epoce, z 1,23 do 0,30 na zbiorze uczącym. Strata testowa osiąga minimum 0,163 w epoce 74, a następnie nieznacznie rośnie, do 0,171 w ostatniej epoce, choć strata ucząca nadal maleje.



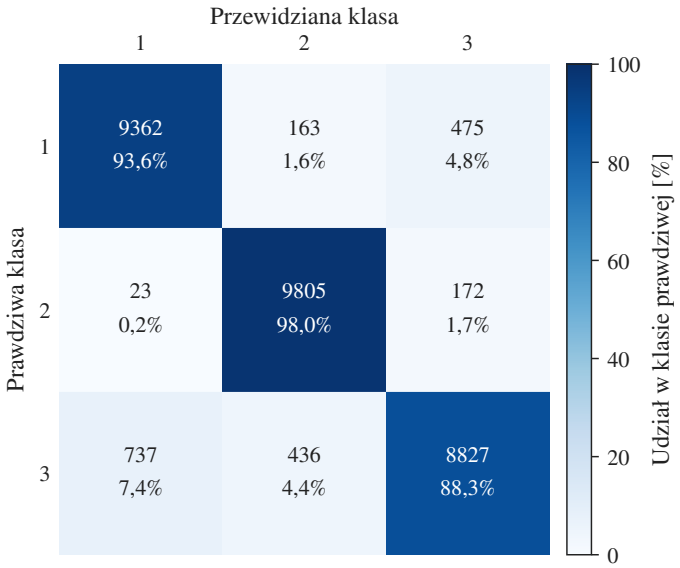
Rysunek 5: Strata (entropia krzyżowa) w trakcie uczenia na zbiorze `three_gauss`. Górny wykres: strata paczek w kolejnych iteracjach ze średnią kroczącą z jednej epoki. Dolny wykres: strata na zbiorach uczącym i testowym po kolejnych epokach.

Dokładność po kolejnych epokach przedstawiono na Rysunku 6. Już po pierwszej epoce wynosi ona 91,87% na zbiorze uczącym i 91,93% na testowym. Dokładność testowa osiąga maksimum 93,76% w epoce 34, a na końcu uczenia wynosi 93,31%, przy 94,67% na zbiorze uczącym.



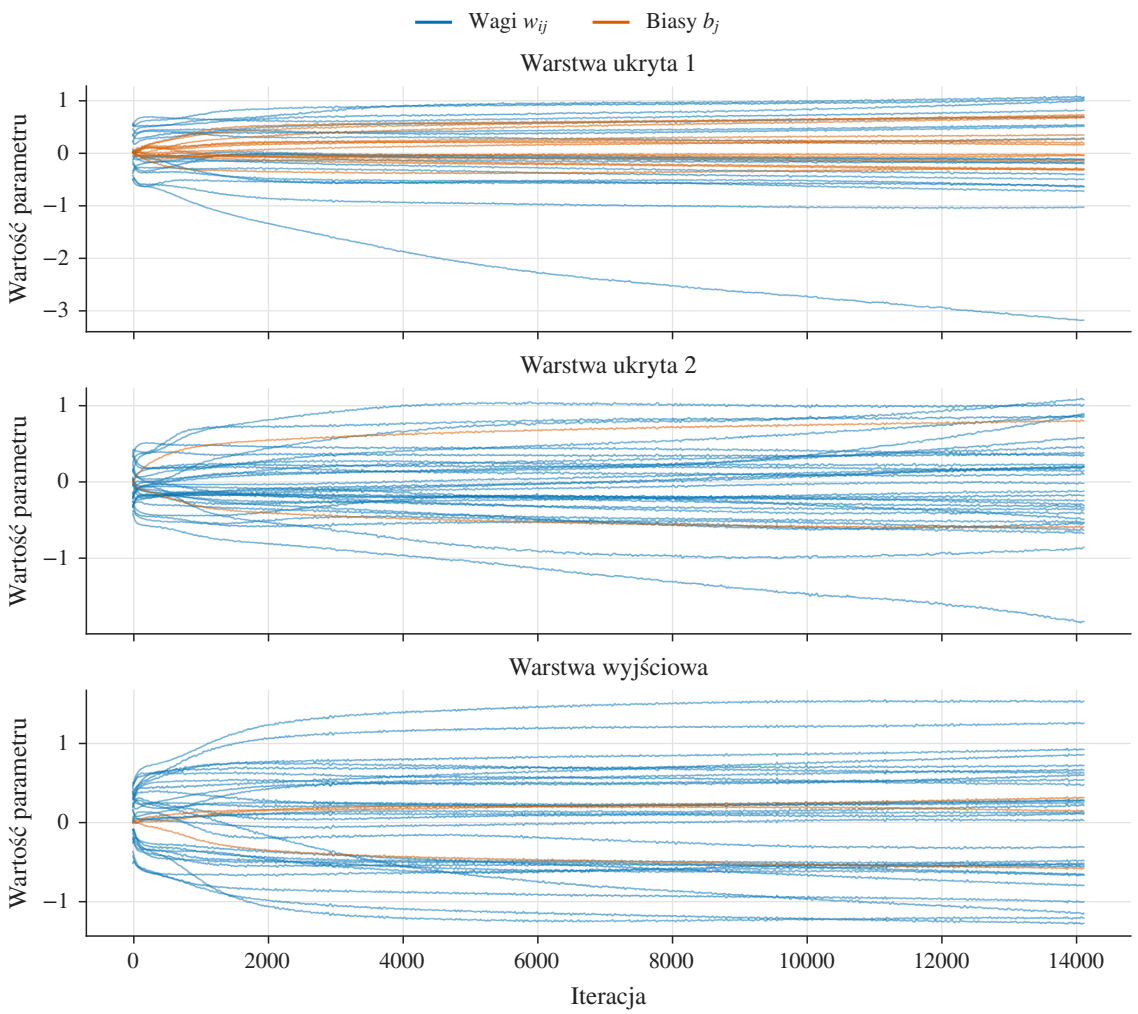
Rysunek 6: Dokładność na zbiorach uczącym i testowym po kolejnych epokach uczenia na zbiorze `three_gauss`.

Macierz pomyłek na zbiorze testowym przedstawiono na Rysunku 7. Najlepiej rozpoznawana jest klasa 2 (98,05% poprawnych decyzji), której skupisko w większości leży z dala od pozostałych (Rysunek 1). Najsłabiej rozpoznawana jest klasa 3 (88,27%), najczęściej mylona z klasą 1 (737 próbek, czyli 7,37%).



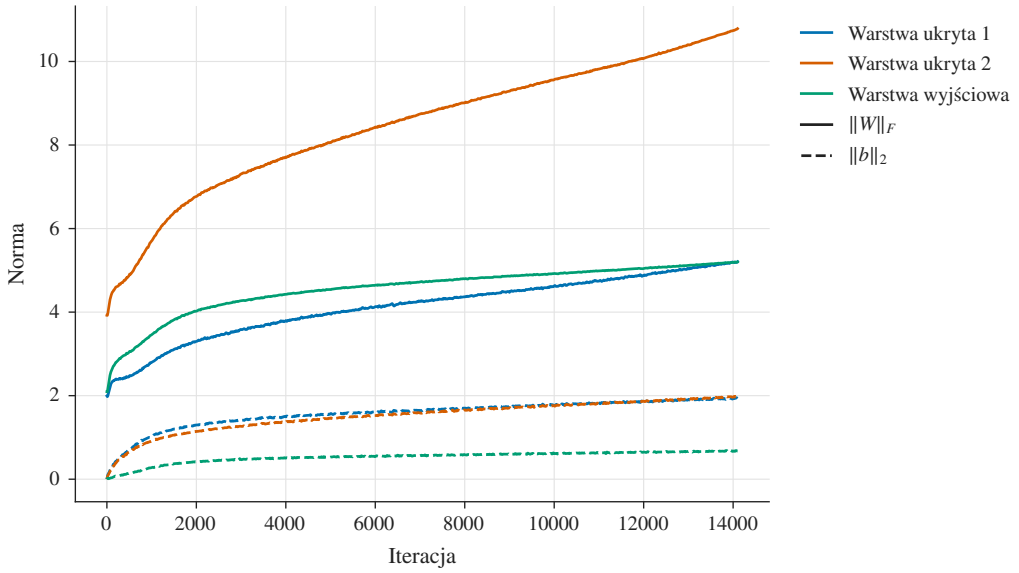
Rysunek 7: Macierz pomyłek na zbiorze testowym `three_gauss`: liczba próbek i odsetek w wierszu klasy prawdziwej.

Ewolucję wartości wag przedstawiono na Rysunku 8. Biasy startują od zera, a wagi od wartości wylosowanych przy inicjalizacji. Najszybciej parametry zmieniają się w pierwszych kilkuset iteracjach, gdy sieć ustala przybliżony kształt granic. Później zmieniają się wolniej, ale nie zatrzymują się, a drobne wahania wynikają z różnic między kolejnymi paczkami.



Rysunek 8: Wartości 32 zapisywanych parametrów każdej warstwy w kolejnych iteracjach uczenia na zbiorze `three_gauss`.

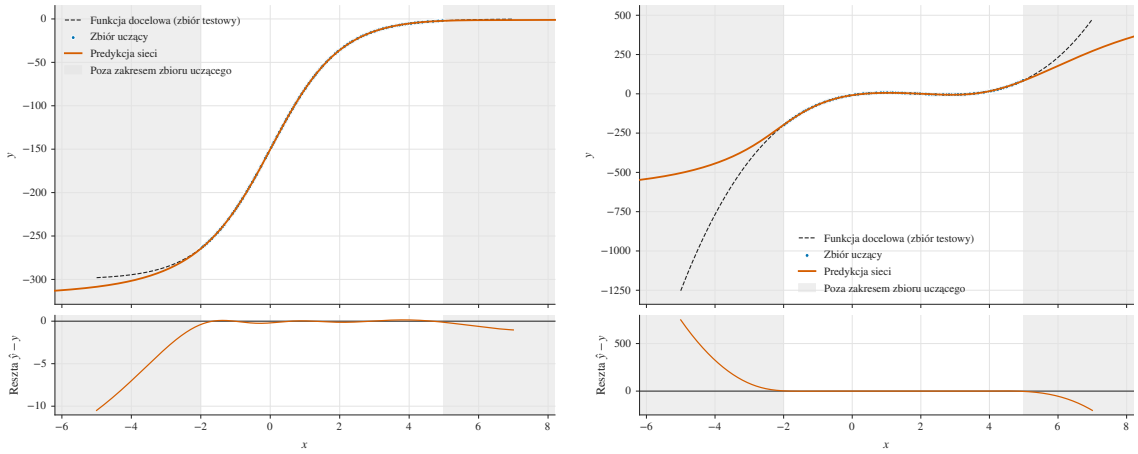
Zbiorczy obraz tych zmian dają normy wag i biasów, przedstawione na Rysunku 9. Norma Frobeniusa $\|W\|_F = \sqrt{\sum_{i,j} w_{ij}^2}$ jest odpowiednikiem normy euklidesowej dla macierzy [1, rozdz. 2.5], więc mierzy łączną wielkość wag warstwy. Norma Frobeniusa wag drugiej warstwy ukrytej rośnie z 3,92 do 10,79, a pozostałych warstw z około 2 do około 5. Wzrost trwa do końca uczenia, co odpowiada powolnemu dryfowi parametrów widocznemu na Rysunku 8.



Rysunek 9: Norma Frobeniusa macierzy wag $\|W\|_F$ i norma euklidesowa wektora biasów $\|b\|_2$ każdej warstwy w kolejnych iteracjach uczenia na zbiorze `three_gauss`.

5.2 Regresja

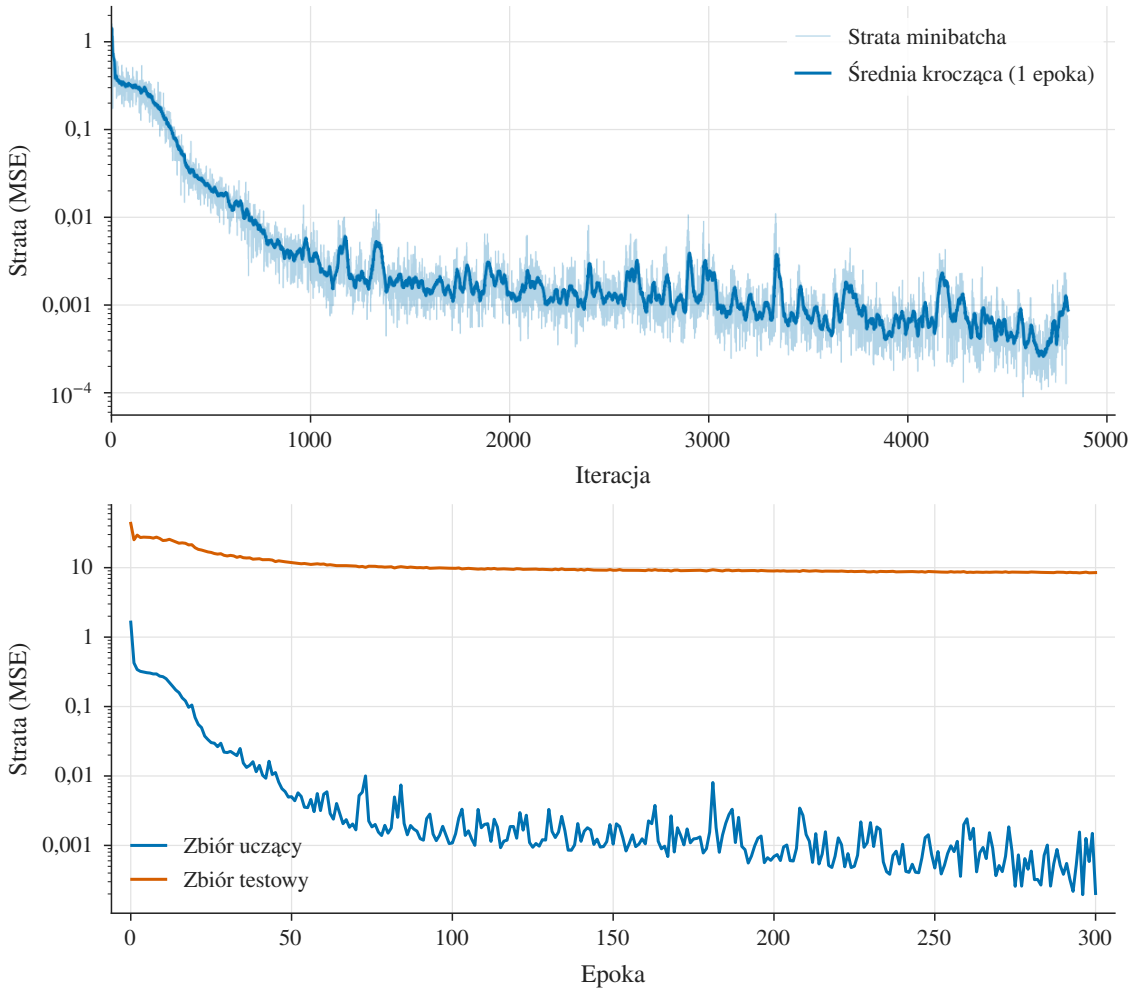
Wyniki regresji pokazano na Rysunku 10. W zakresie zbioru uczącego obie sieci dopasowują funkcję docelową z resztą bliską zeru. Poza nim odpowiedź sieci z funkcją tanh wypłaszcza się. Dla zbioru `activation`, którego funkcja również się nasyca, reszta pozostaje niewielka. Dla zbioru `cube`, którego funkcja szybko rośnie co do wartości bezwzględnej, reszta rośnie z odległością od danych uczących. Zależność tę zbadano dokładniej w rozdziale 8.



(a) `activation` (b) `cube`

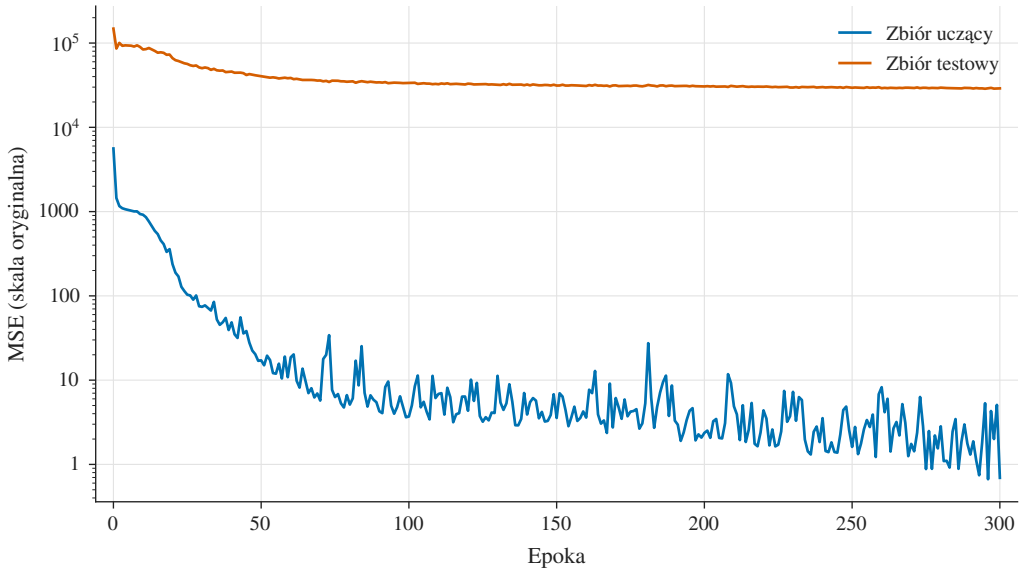
Rysunek 10: Punkty uczące, funkcja docelowa i odpowiedź sieci (górne wykresy) oraz reszty na zbiorze testowym (dolne wykresy). Szare pasy oznaczają wejścia spoza zakresu zbioru uczącego.

Przebieg błędu na zbiorze `cube` przedstawiono na Rysunku 11. Strata jest liczona na celach po standaryzacji. Średnia krocząca straty paczek maleje w pierwszym tysiącu iteracji o ponad dwa rzędy wielkości, z 0,59 do 0,0031, a potem zmniejsza się już powoli, z wyraźnymi wahaniami. Strata testowa pozostaje o kilka rzędów wielkości większa od uczącej (w ostatniej epoce 8,50 wobec $2{,}0 \cdot 10^{-4}$), bo obejmuje punkty spoza zakresu uczącego.



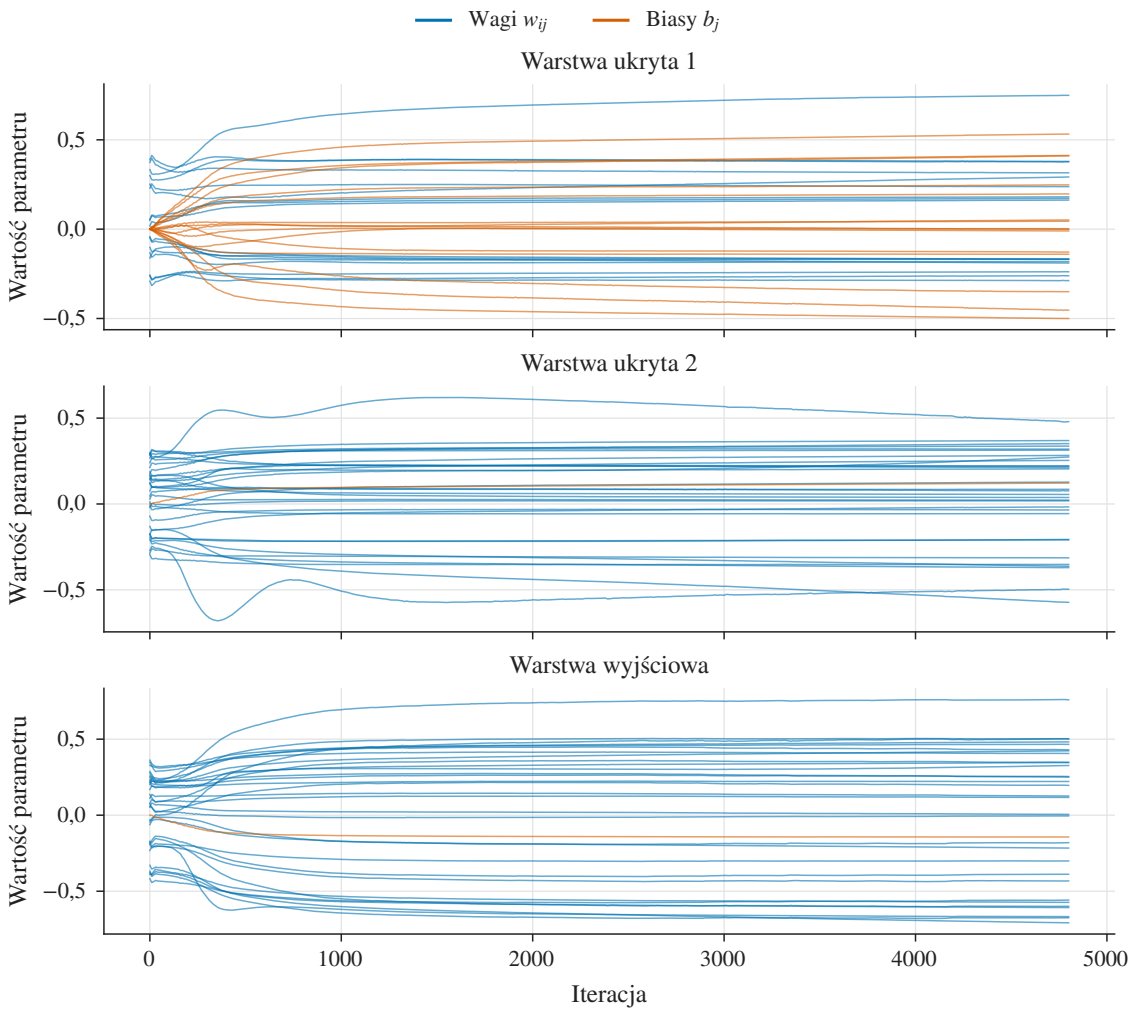
Rysunek 11: Strata (MSE na celach po standaryzacji) w trakcie uczenia na zbiorze `cube`. Górny wykres: strata paczek w kolejnych iteracjach ze średnią kroczącą z jednej epoki. Dolny wykres: strata na zbiorach uczącym i testowym po kolejnych epokach.

Błąd średniokwadratowy w oryginalnej skali odpowiedzi przedstawiono na Rysunku 12. Na końcu uczenia wynosi on 0,693 na zbiorze uczącym i 28 991 na testowym. Dla zbioru `activation` różnica jest mniejsza: 0,0156 na zbiorze uczącym i 9,21 na testowym. Błąd testowy dla tego zbioru był najmniejszy w epoce 31 (2,02), a później rósł, mimo że błąd uczący nadal malał.

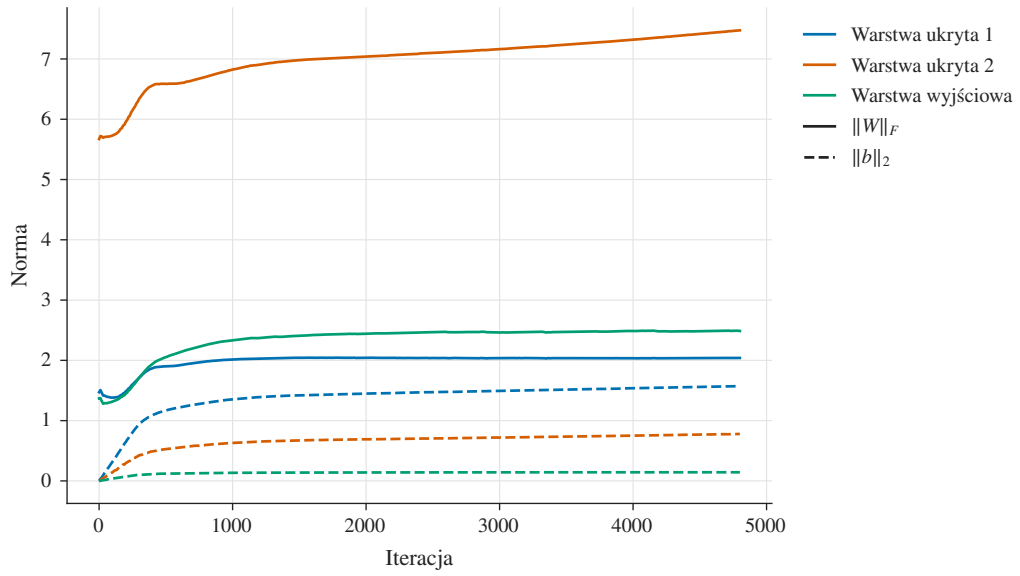


Rysunek 12: Błąd średniokwadratowy w oryginalnej skali odpowiedzi na zbiorach uczącym i testowym po kolejnych epokach uczenia na zbiorze `cube`.

Ewolucję wag sieci uczonej na zbiorze `cube` przedstawiono na Rysunkach 13 i 14. W odróżnieniu od klasyfikacji zbioru `three_gauss` większość zmian zachodzi w pierwszym tysiącu iteracji. Norma wag pierwszej warstwy ukrytej rośnie w tym czasie z 1,47 do 2,01, a do końca uczenia już tylko do 2,04. Trajektorie poszczególnych parametrów są przy tym gładsze niż na Rysunku 8.



Rysunek 13: Wartości 32 zapisywanych parametrów każdej warstwy w kolejnych iteracjach uczenia na zbiorze `cube`.



Rysunek 14: Norma Frobeniusa macierzy wag $\|W\|_F$ i norma euklidesowa wektora biasów $\|b\|_2$ każdej warstwy w kolejnych iteracjach uczenia na zbiorze `cube`.

6 Metodyka eksperymentów

W rozdziale opisano plan badań, wspólną konfigurację bazową sieci, miary i statystyki używane do porównywania wariantów oraz założenia planu, które należy uwzględnić przy interpretacji wyników przedstawionych w rozdziałach 7–9.

6.1 Plan badań

Każde badanie zmienia jeden czynnik względem wspólnej konfiguracji bazowej, a pozostałe ustawienia pozostawia bez zmian (ang. *one-factor-at-a-time*). Badane czynniki i ich poziomy zestawiono w Tabeli 8. Funkcja aktywacji zmienia się razem z inicjalizacją wag dobraną do niej zgodnie z podrozdziałem 3.6. Obie funkcje straty klasyfikacji są stosowane z funkcją softmax na wyjściu, a obie funkcje straty regresji z identycznością (Tabela 4), więc w badaniu strat zmienia się tylko minimalizowana wielkość.

Tabela 8: Badane czynniki i ich poziomy. Architekturę podano jako liczby neuronów kolejnych warstw ukrytych.

Czynnik	Poziomy	Liczba poziomów
funkcja aktywacji	funkcja sigmoidalna i tanh z inicjalizacją Xaviera, ReLU i leaky ReLU z $\alpha = 0{,}01$ z inicjalizacją He	4
architektura	brak warstw ukrytych, 4, 16, 64, 16-16, 64-64, 16-16-16, 16-16-16-16, 64-64-64-64	9
funkcja straty	klasyfikacja: entropia krzyżowa i MSE, regresja: MSE i MAE	2

Każdy czynnik zbadano na każdym z czterech zbiorów, co daje 12 badań. Drugą osią każdego badania jest liczność zbioru uczącego, z poziomami 100, 500, 1000 i 10 000 odpowiadającymi dostarczonym plikom (Tabela 1). W zbiorze `three_gauss` liczność dotyczy jednej klasy. Wariant badania to kombinacja poziomu czynnika i liczności, a każdy wariant uczono dla 10 ziaren generatora, od 1 do 10. Na jeden zbiór przypada więc 60 wariantów i 600 eksperymentów, a na wszystkie badania 2400 eksperymentów. Wszystkie zakończyły się po zadanej liczbie epok, bez rozbieżności uczenia. Położenie konfiguracji badań podano w dodatku A.

Liczbę parametrów sieci o badanych architekturach przedstawiono w Tabeli 9. Zależy ona od liczby wejść i wyjść, więc jest inna dla każdego zbioru klasyfikacji i wspólna dla obu zbiorów regresji. Największa sieć, 64-64-64-64, ma od 12 673 do 12 867 parametrów, czyli wielokrotnie więcej niż próbek w najmniejszych zbiorach uczących.

Tabela 9: Liczba parametrów (wag i biasów) sieci o badanych architekturach. Sieć `simple` ma 2 wejścia i 2 wyjścia, `three_gauss` 2 wejścia i 3 wyjścia, a sieci regresji 1 wejście i 1 wyjście.

Warstwy ukryte	`simple`	`three_gauss`	regresja
brak	6	9	2
4	22	27	13
16	82	99	49
64	322	387	193
16-16	354	371	321
64-64	4482	4547	4353
16-16-16	626	643	593
16-16-16-16	898	915	865
64-64-64-64	12 802	12 867	12 673

6.2 Konfiguracja bazowa

Konfigurację bazową zestawiono w Tabeli 10. Bazowa sieć regresji ma dwie warstwy ukryte po 32 neurony. Architektura 32-32 nie należy do poziomów badania architektur, które są wspólne dla wszystkich zbiorów. Wariant bazowy występuje więc w trzech badaniach każdego zbioru klasyfikacji i w dwóch badaniach każdego zbioru regresji. W każdym z tych badań daje identyczne wyniki dla tych samych ziaren.

Tabela 10: Konfiguracja bazowa badań. Każde badanie zmienia jeden z tych parametrów zgodnie z Tabelą 8 oraz liczność zbioru uczącego i liczbę epok zgodnie z Tabelą 11.

Parametr	Klasyfikacja	Regresja
warstwy ukryte	16-16	32-32
aktywacja warstw ukrytych	tanh	tanh
inicjalizacja wag	Xavier	Xavier
biasy	tak	tak
aktywacja wyjściowa	softmax	identyczność
funkcja straty	entropia krzyżowa	MSE
optymalizator	Adam	Adam
współczynnik uczenia η	0,003	0,003
rozmiar paczki	32	32
normalizacja wejść	standaryzacja	standaryzacja
normalizacja celu	—	standaryzacja
zbiór walidacyjny	brak	brak
zbiór testowy	`test.10000`	1201 punktów

Optymalizator Adam działa z domyślnymi wartościami β_1, β_2 i ϵ podanymi w podrozdziale 3.5. Liczbę epok dobrano do liczności zbioru uczącego zgodnie z Tabelą 11. Epoka zbioru o n próbkach składa się z $\lceil n/32 \rceil$ paczek, więc uczenie przez E epok wykonuje $E\lceil n/32 \rceil$ iteracji. Liczba epok maleje wolniej, niż rośnie liczność, dzięki czemu nawet dla najmniejszych zbiorów sieć wykonuje kilka tysięcy iteracji. Metryki każdego eksperymentu pochodzą z ostatniej epoki.

Tabela 11: Liczba epok i iteracji w zależności od liczności zbioru uczącego.

Liczność	Epoki	Iteracje	
		`simple`, `activation`, `cube`	`three_gauss`
100	1000	4000	10 000
500	300	4800	14 100
1000	200	6400	18 800
10 000	30	9390	28 140

6.3 Miary jakości i statystyki

W klasyfikacji porównywaną miarą jest dokładność testowa (2). Dla każdego wariantu podaje się jej średnią i próbkowe odchylenie standardowe po 10 ziarnach. W regresji porównywaną miarą jest błąd średniokwadratowy (1) na zbiorze testowym, liczony w oryginalnej skali odpowiedzi, czyli po odwróceniu standaryzacji celu. Rozkład MSE po ziarnach jest silnie prawostronnie skośny, bo pojedyncze ziarno może dać błąd o rząd wielkości większy od pozostałych. Na przykład dla sieci 16-16-16 uczonej na 10 000 próbkach zbioru `activation` mediana MSE w zakresie zbioru uczącego wynosi 0,0054, a średnia 0,54. Średnią zawyża głównie jedno ziarno z błędem 4,64. Średnia opisuje wtedy to jedno ziarno, a nie typowy wynik, a średnia pomniejszona o odchylenie bywa ujemna. Dlatego dla MSE podaje się medianę i kwartyle Q_1 i Q_3 (z interpolacją liniową), które są zawsze dodatnie i pozwalają na rysowanie w skali logarytmicznej.

Zbiór testowy regresji pokrywa przedział $[-5, 7]$, szerszy niż zakres wejść zbioru uczącego (podrozdział 2.2). MSE na całym zbiorze testowym łączy więc dwie różne własności sieci: dopasowanie do funkcji w obszarze, z którego pochodzą dane uczące, i przebieg funkcji sieci poza nim. Dlatego dla każdego eksperymentu MSE liczy się również osobno dla punktów testowych leżących w zakresie $[x_{\min}, x_{\max}]$ wejść jego zbioru uczącego (interpolacja) i poza tym zakresem (ekstrapolacja):

$$\mathrm{MSE}_{\mathrm{w}} = \frac{1}{|\mathcal{T}_{\mathrm{w}}|} \sum_{i \in \mathcal{T}_{\mathrm{w}}} (\hat{y}_i - y_i)^2, \qquad \mathcal{T}_{\mathrm{w}} = \{\, i : x_{\min} \le x_i \le x_{\max} \,\}, \tag{15}$$

$$\mathrm{MSE}_{\mathrm{poza}} = \frac{1}{|\mathcal{T}_{\mathrm{poza}}|} \sum_{i \in \mathcal{T}_{\mathrm{poza}}} (\hat{y}_i - y_i)^2, \qquad \mathcal{T}_{\mathrm{poza}} = \{1, \ldots, m\} \setminus \mathcal{T}_{\mathrm{w}}, \tag{16}$$

gdzie i przebiega punkty zbioru testowego, a $x_{\min}$ i $x_{\max}$ to najmniejsze i największe wejście zbioru uczącego danego eksperymentu. Miary (15) i (16) podsumowuje się po ziarnach tak samo jak MSE.

Wyniki badań przedstawiono na wykresach kilku rodzajów, wspólnych dla wszystkich badań:

- wykres wyników ziaren: końcowa metryka testowa każdego ziarna jako punkt oraz średnia z odchyleniem albo mediana z kwartylami dla każdego wariantu,
- mapa ciepła: ta sama statystyka w komórkach tabeli o wierszach odpowiadających licznościom i kolumnach odpowiadających poziomom czynnika,
- krzywe uczenia: metryka testowa po kolejnych epokach jako średnia z pasem odchylenia albo mediana z pasem kwartyli, z osobnym panelem dla każdej liczności (oś dokładności pomija pierwsze 10% epok, w których dokładność szybko rośnie od poziomu losowego),
- krzywe straty uczącej w tej samej postaci, rysowane tylko w badaniach, w których wszystkie warianty minimalizują tę samą funkcję straty,
- galerie: granice decyzyjne albo krzywe regresji jednego eksperymentu każdego wariantu, tego, którego metryka testowa jest najbliższa medianie po ziarnach (przy równej odległości wybiera się mniejsze ziarno).

Różnice między wariantami porównywalne z odchyleniem standardowym między ziarnami lub mniejsze od rozstępu kwartylowego opisuje się jako słabe. Za istotne uważa się tylko efekty wyraźnie większe od rozrzutu wyników między ziarnami.

6.4 Założenia i ograniczenia planu

Plan badań ma kilka cech, które ograniczają wnioski z porównań:

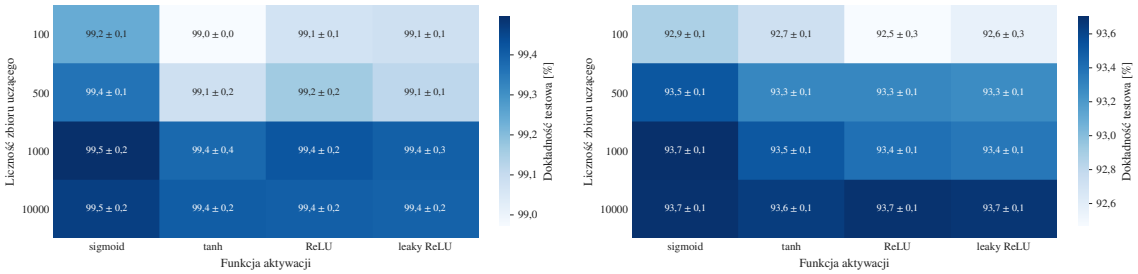
- budżet uczenia rośnie z licznością zbioru (Tabela 11), więc efekt liczności obejmuje także większą liczbę iteracji optymalizatora,
- pliki różnych liczności są rozłączne, a mniejszy zbiór nie jest podzbiorem większego, więc zmiana liczności zmienia też konkretną próbkę danych,
- współczynnik uczenia jest jeden dla wszystkich wariantów, wybrany w krótkim badaniu wstępnym jako kompromis między stabilnością głębokich sieci a zbieżnością płytkich, więc wyniki części architektur mogłyby się poprawić przy innym współczynniku uczenia,
- badania nie używają zbioru walidacyjnego ani wczesnego zatrzymania (ang. *early stopping*) [1, rozdz. 7.8], więc warianty porównuje się przy stałym budżecie, a przeuczenie przy małych zbiorach jest częścią wyniku.

7 Wyniki klasyfikacji

W rozdziale przedstawiono wyniki sześciu badań klasyfikacji, po trzy dla zbiorów `simple` i `three_gauss`, w kolejności czynników z Tabeli 8. Miarą jest dokładność testowa, podawana jako średnia i odchylenie standardowe po 10 ziarnach (podrozdział 6.3).

7.1 Funkcje aktywacji

Wyniki badania funkcji aktywacji przedstawiono na Rysunku 15. W zbiorze `simple` funkcja sigmoidalna jest najlepsza przy każdej liczności, ale jej przewaga nad najsłabszą funkcją wynosi co najwyżej 0,28 punktu procentowego, a przy licznościach 1000 i 10 000 jest mniejsza niż zmiana wyniku wywołana samą zmianą ziarna (odchylenie standardowe od 0,16 do 0,37 punktu procentowego). Różnice między funkcjami aktywacji w tym zbiorze są więc słabe.



(a) `simple` (b) `three_gauss`

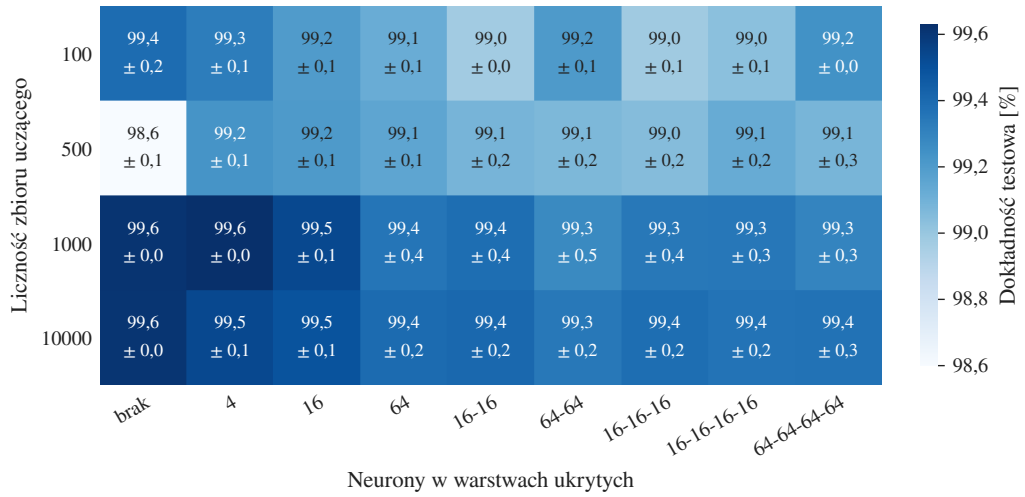
Rysunek 15: Dokładność testowa w zależności od funkcji aktywacji warstw ukrytych i liczności zbioru uczącego: średnia i odchylenie standardowe po 10 ziarnach.

W zbiorze `three_gauss` funkcja aktywacji ma znaczenie tylko dla małych zbiorów uczących. Przy liczności 100 najlepsza jest funkcja sigmoidalna (92,88%), a najsłabsza ReLU (92,47%). Funkcje ReLU i leaky ReLU mają przy tym największy rozrzut między ziarnami i najwyższą dokładność na zbiorze uczącym (97,00% i 97,17%), a funkcja sigmoidalna najniższą (95,27%). Przy liczności 10 000 wszystkie funkcje osiągają od 93,64% do 93,69%, czyli mniej, niż zmienia wynik sama zmiana ziarna.

Sieć z funkcją ReLU jest funkcją kawałkami liniową, więc łatwo tworzy ostre załamania granicy wokół pojedynczych punktów uczących. Przy małym zbiorze sprzyja to dopasowaniu do punktów z obszaru nakładania się klas, co widać jako wyższą dokładność uczącą i niższą testową. Funkcja sigmoidalna, najsłabiej dopasowana do zbioru uczącego, przeucza się najmniej. Różnice są jednak niewielkie i dotyczą tylko najmniejszych zbiorów.

7.2 Architektura sieci

Wyniki badania architektur dla zbioru `simple` przedstawiono na Rysunku 16. Sieć bez warstw ukrytych jest najlepsza przy licznościach 100 i 10 000 (99,39% i 99,60%), a przy liczności 1000 ustępuje tylko sieci z 4 neuronami, o 0,02 punktu procentowego. Ma też najmniejszy rozrzut między ziarnami przy licznościach 1000 i 10 000. Większe sieci nie poprawiają wyniku, a ich rozrzut rośnie.

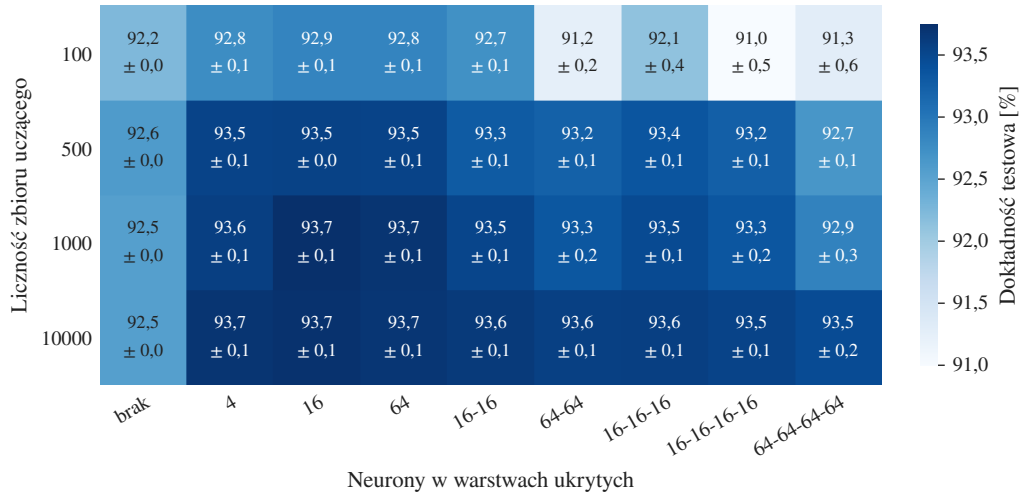


Rysunek 16: Dokładność testowa na zbiorze `simple` w zależności od architektury i liczności zbioru uczącego: średnia i odchylenie standardowe po 10 ziarnach.

Wyjątkiem jest liczność 500, przy której sieć bez warstw ukrytych osiąga tylko 98,60%, wobec 99,05–99,23% dla sieci z warstwami ukrytymi. Jej dokładność na zbiorze uczącym wynosi 98,92%, więc ten konkretny plik uczący nie jest rozdzielany prostą bez błędów, a kilka punktów po niewłaściwej stronie granicy przesuwa wyznaczoną prostą. Pliki różnych liczności są rozłączne (podrozdział 6.4), więc ten efekt dotyczy tylko jednej próbki danych.

Sieć bez warstw ukrytych z funkcją softmax na wyjściu jest modelem regresji logistycznej, którego granice decyzyjne są prostymi [3, rozdz. 4.3]. Klasy zbioru `simple` są rozdzielone prostą (podrozdział 2.2), więc ten model ma dokładnie potrzebną postać granicy. Większe sieci mogą przedstawić bardziej złożone granice, ale przy granicy prostej nie mają czego poprawić, a pozwalają dopasować granicę do pojedynczych punktów przy prostej. Sposób tego dopasowania zależy od ziarna, stąd większy rozrzut.

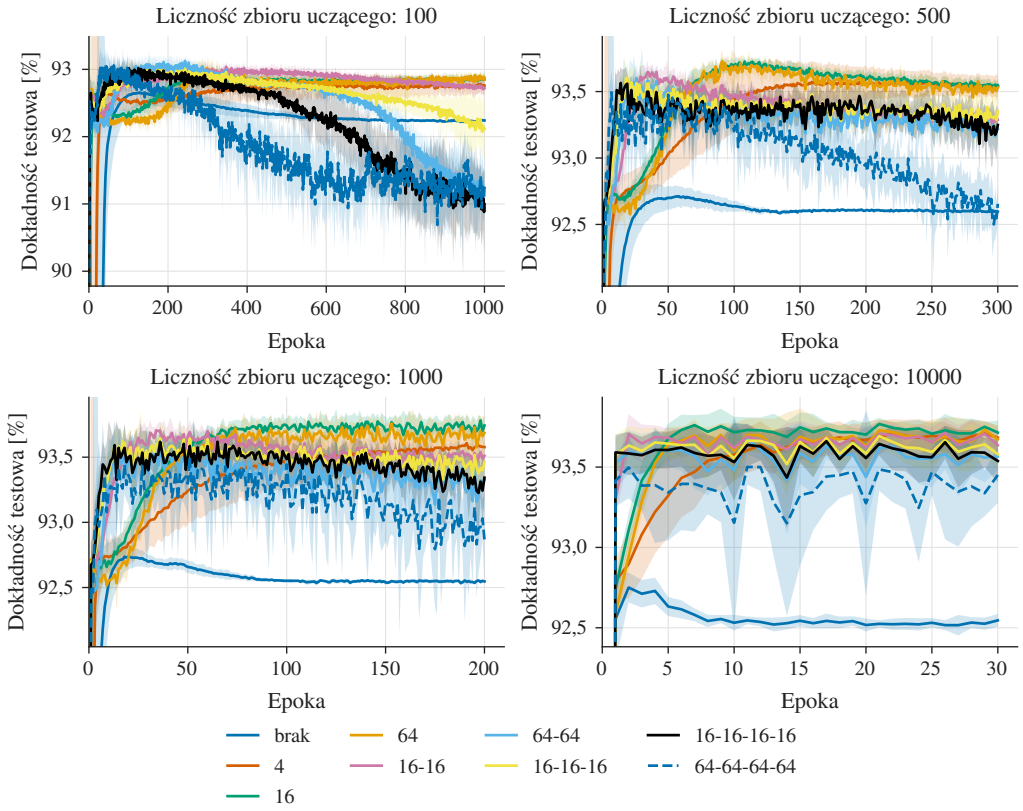
Wyniki dla zbioru `three_gauss` przedstawiono na Rysunku 17. Sieć bez warstw ukrytych osiąga od 92,24% do 92,60%, z odchyleniem nie większym niż 0,04 punktu procentowego. Już jedna warstwa z 4 neuronami poprawia wynik o około punkt procentowy (93,54% przy liczności 500 i 93,68% przy 10 000). Najwyższa średnia w całym badaniu wynosi 93,74% (jedna warstwa z 16 neuronami, liczność 1000). Żadna architektura nie przekracza tego poziomu, co jest zgodne z nakładaniem się klas, w którego obszarze myli się nawet klasyfikator Bayesa (podrozdział 2.1).



Rysunek 17: Dokładność testowa na zbiorze `three_gauss` w zależności od architektury i liczności zbioru uczącego: średnia i odchylenie standardowe po 10 ziarnach.

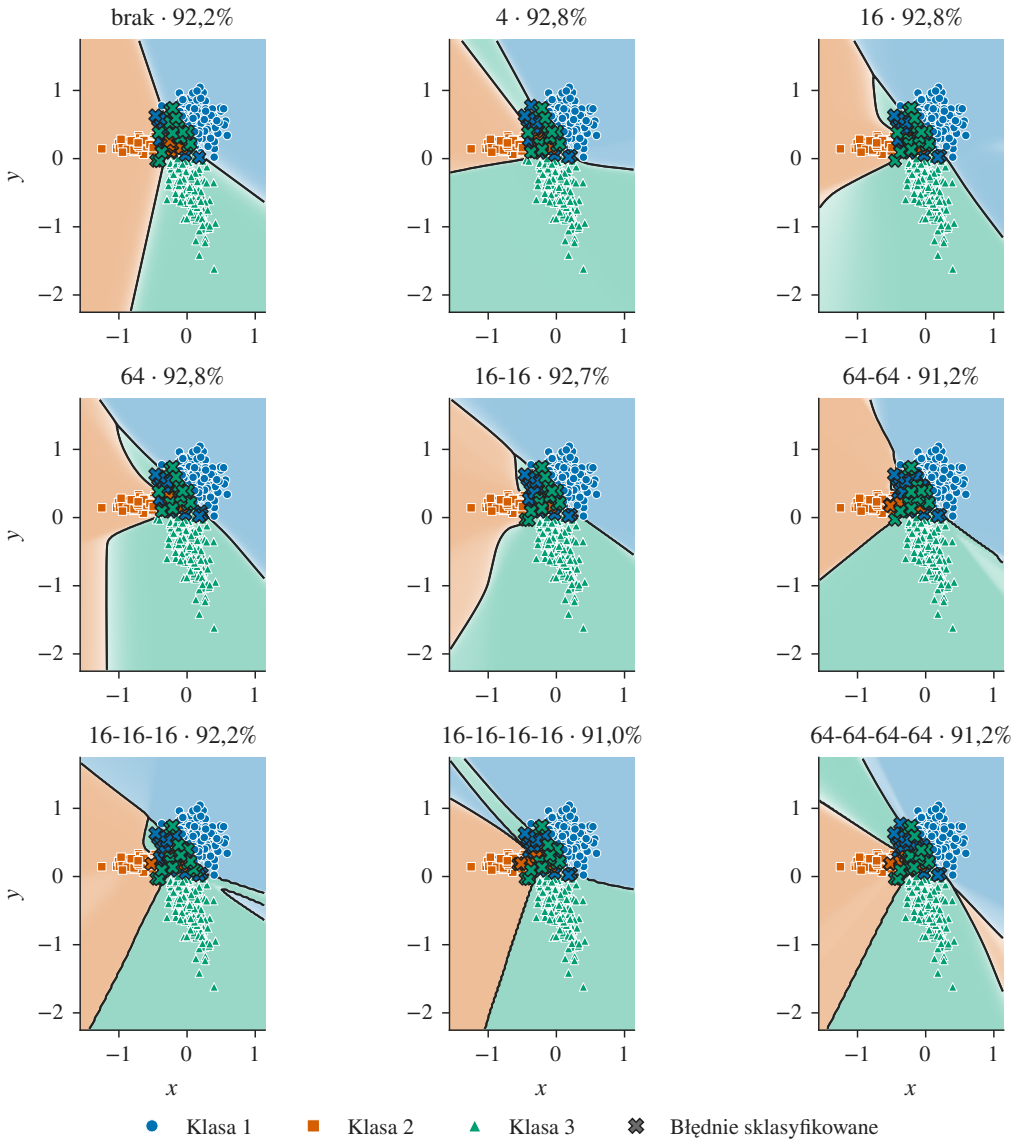
Przy liczności 100 głębokie i szerokie sieci wypadają wyraźnie gorzej od małych. Sieci 64-64, 16-16-16-16 i 64-64-64-64 osiągają od 91,00% do 91,28%, wobec 92,85% dla jednej warstwy z 16 lub 64 neuronami, choć na zbiorze uczącym przekraczają 98%. Przebieg tego zjawiska pokazują krzywe uczenia na Rysunku 18. Przy liczności 100 średnia dokładność testowa dużych sieci najpierw rośnie

do około 93%, np. dla sieci 64-64 do 93,13% w epoce 243, a potem spada do wartości końcowych podanych wyżej. Przy liczności 10 000 sieć 64-64-64-64 wyróżnia się wahaniami dokładności między kolejnymi epokami. Największa zmiana między epokami w pojedynczym uczeniu wynosi dla niej średnio 0,79 punktu procentowego, a dla pozostałych architektur od 0,11 do 0,49.



Rysunek 18: Dokładność testowa na zbiorze `three_gauss` po kolejnych epokach dla każdej architektury i liczności zbioru uczącego: średnia po 10 ziarnach z pasem jednego odchylenia standardowego.

Granice decyzyjne sieci uczonych na najmniejszym zbiorze pokazano na Rysunku 19. Dla każdej architektury pokazano jedno z 10 uczeń: to, którego dokładność testowa jest najbliższa medianie po ziarnach (podrozdział 6.3). Sieć bez warstw ukrytych wyznacza granice proste, a małe sieci z jedną warstwą ukrytą gładkie krzywe. Granice głębokich sieci są nieregularne, a poza obszarem danych pojawiają się u nich dodatkowe wąskie obszary klas, których nie uzasadnia żaden punkt.

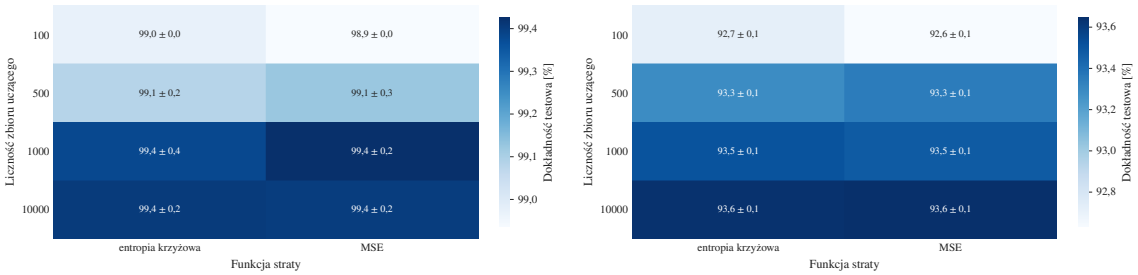


Rysunek 19: Obszary decyzyjne uczenia o dokładności testowej najbliższej medianie po ziarnach, dla każdej architektury przy liczności 100 na klasę, wraz z tą dokładnością. Punkty to równomiernie wybrana próbka zbioru testowego.

Wyniki dla zbioru `three_gauss` wyjaśniają dwa mechanizmy. Optymalne granice między skupiskami o różnych kształtach i rozrzutach nie są prostymi. Dla rozkładów normalnych o różnych macierzach kowariancji są to krzywe stopnia drugiego [3, rozdz. 4.2]. Sieć bez warstw ukrytych nie może ich wyrazić, a już mała warstwa ukryta je przybliża. Drugim mechanizmem jest przeuczenie przy małym zbiorze [1, rozdz. 5.2]. Sieć 64-64-64-64 ma 12 867 parametrów (Tabela 9) przy 300 próbkach uczących. Po osiągnięciu maksimum dokładności testowej dopasowuje granice do pojedynczych punktów z obszaru nakładania się klas, czyli do szumu, a nie do struktury danych. Przy 10 000 próbkach na klasę takich punktów jest zbyt wiele, by sieć mogła je zapamiętać, więc różnice między architekturami zanikają.

7.3 Funkcje straty

Wyniki badania funkcji straty przedstawiono na Rysunku 20. Entropia krzyżowa i MSE, obie z funkcją softmax na wyjściu, dają praktycznie tę samą dokładność. Różnica średnich nie przekracza 0,05 punktu procentowego w zbiorze `simple` i 0,09 punktu procentowego w zbiorze `three_gauss`, czyli jest mniejsza niż rozrzut między ziarnami. Żadna z funkcji straty nie ma więc przewagi.



(a) `simple` (b) `three_gauss`

Rysunek 20: Dokładność testowa w zależności od funkcji straty i liczności zbioru uczącego: średnia i odchylenie standardowe po 10 ziarnach.

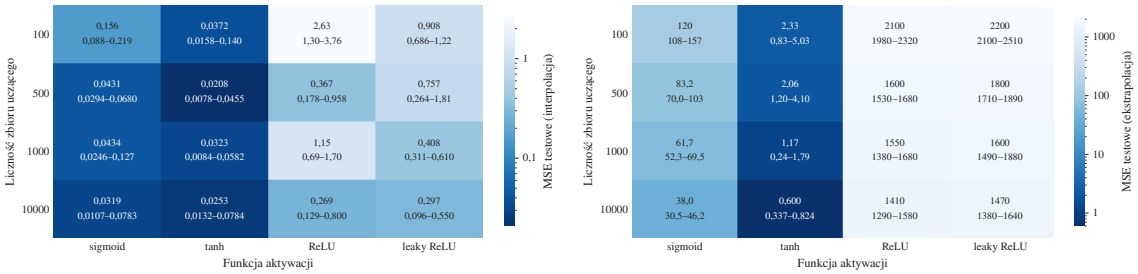
Teoretyczną przewagą entropii krzyżowej jest gradient (8), proporcjonalny do błędu niezależnie od nasycenia funkcji softmax. Dla MSE gradient przechodzi przez pochodną funkcji softmax i maleje, gdy sieć jest pewna błędnej decyzji, co może spowalniać uczenie [1, rozdz. 6.2.2]. W badanych zadaniach ta różnica się nie ujawnia. Klasy są tylko dwie lub trzy, a metoda Adam normalizuje skalę gradientu każdego parametru [5], więc obie straty prowadzą do tych samych granic.

8 Wyniki regresji

W rozdziale przedstawiono wyniki sześciu badań regresji, po trzy dla zbiorów `activation` i `cube`. Miarą jest MSE testowe w oryginalnej skali odpowiedzi, podawane jako mediana i kwartyle po 10 ziarnach, osobno dla punktów w zakresie zbioru uczącego (15) i poza nim (16).

8.1 Funkcje aktywacji

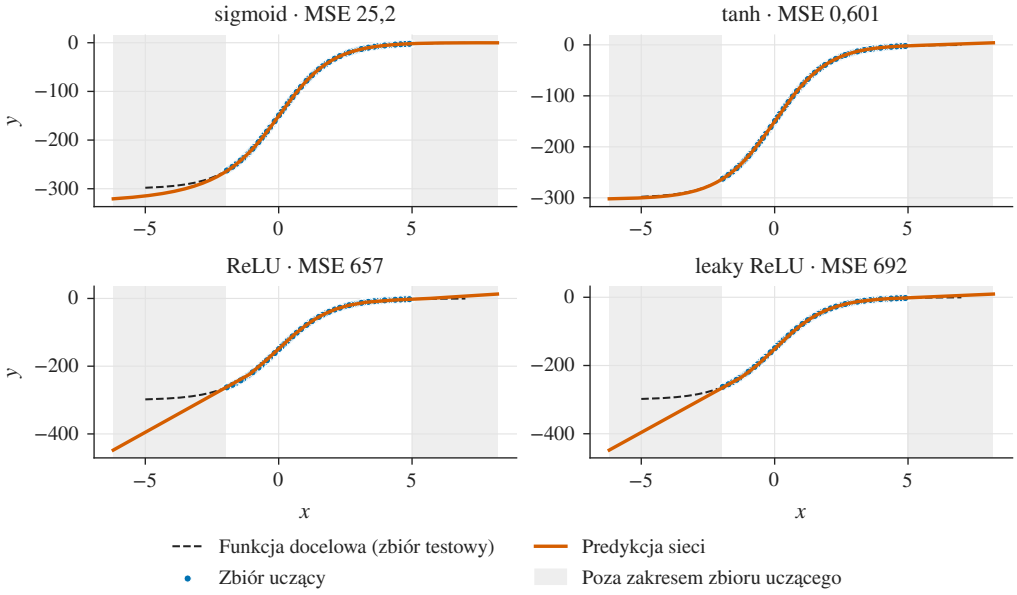
Wyniki badania funkcji aktywacji dla zbioru `activation` przedstawiono na Rysunku 21. W zakresie zbioru uczącego najlepsza przy każdej liczności jest funkcja tanh, z medianą MSE od 0,0208 do 0,0372, a ReLU i leaky ReLU osiągają od 0,269 do 2,63. Poza zakresem różnice sięgają trzech rzędów wielkości. Mediana MSE wynosi od 0,600 do 2,33 dla funkcji tanh, od 38,0 do 120 dla funkcji sigmoidalnej i od 1410 do 2200 dla ReLU i leaky ReLU.



(a) w zakresie zbioru uczącego (b) poza zakresem zbioru uczącego

Rysunek 21: MSE testowe na zbiorze `activation` w zależności od funkcji aktywacji warstw ukrytych i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach.

Źródło tych różnic widać na Rysunku 22, który dla każdej funkcji aktywacji pokazuje jedno z 10 uczeń przy liczności 1000: to, którego MSE testowe jest najbliższe medianie po ziarnach. W zakresie danych uczących wszystkie cztery sieci pokrywają się z funkcją docelową. Poza nim sieci z funkcjami tanh i sigmoidalną nasycają się tak jak funkcja docelowa, przy czym sieć z funkcją sigmoidalną na lewym krańcu dochodzi do innego poziomu. Sieci z ReLU i leaky ReLU przedłużają funkcję prostą, która coraz bardziej oddala się od nasyconej funkcji docelowej.

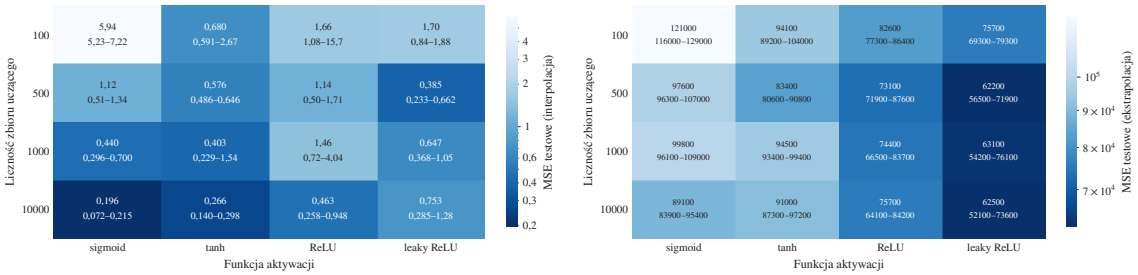


Rysunek 22: Funkcja sieci na zbiorze `activation` przy liczności 1000 dla każdej funkcji aktywacji: uczenie o MSE testowym najbliższym medianie po ziarnach i jego MSE testowe. Szare pasy oznaczają obszar poza zakresem zbioru uczącego.

Daleko od danych pobudzenia neuronów pierwszej warstwy ukrytej rosną co do wartości bezwzględnej liniowo z x. Funkcje tanh i sigmoidalna mają ograniczony zbiór wartości (Tabela 3),

więc ich wyjścia stają się stałe, a z nimi cała funkcja sieci. Sieć z funkcjami ReLU lub leaky ReLU jest funkcją kawałkami liniową, więc daleko od danych jest liniowa [10]. Funkcja docelowa zbioru `activation` nasyca się na obu krańcach (podrozdział 2.2), dlatego przedłużenie stałe pasuje do niej, a przedłużenie liniowe nie.

Wyniki dla zbioru `cube` przedstawiono na Rysunku 23. W zakresie zbioru uczącego przy liczności 100 najlepsza jest funkcja tanh (0,680), a najsłabsza funkcja sigmoidalna (5,94). Przy liczności 10 000 kolejność się zmienia: funkcja sigmoidalna osiąga 0,196, tanh 0,266, a ReLU i leaky ReLU 0,463 i 0,753. Poza zakresem wszystkie sieci mają błędy rzędu 10^4–10^5. Najmniejsze ma leaky ReLU (od 62 200 do 75 700), a największe funkcja sigmoidalna (od 89 100 do 121 000).



(a) w zakresie zbioru uczącego

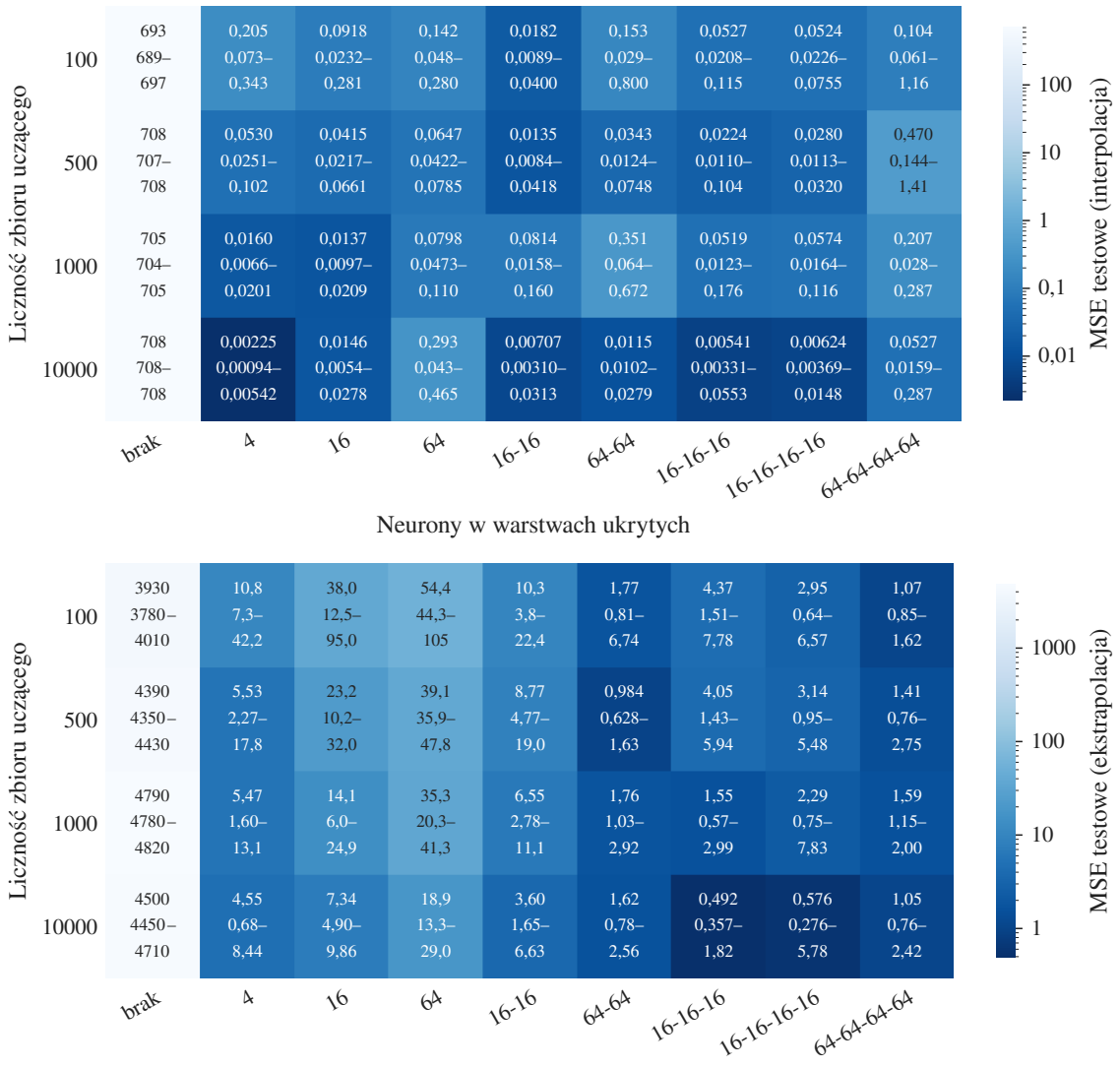
(b) poza zakresem zbioru uczącego

Rysunek 23: MSE testowe na zbiorze `cube` w zależności od funkcji aktywacji warstw ukrytych i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach.

Funkcja docelowa zbioru `cube` na krańcach szybko rośnie co do wartości bezwzględnej, szybciej niż jakakolwiek prosta. Żadna z badanych sieci nie może więc jej przedłużyć, a ranking poza zakresem mówi tylko, której sieci przedłużenie najmniej odbiega od funkcji docelowej. Przedłużenie liniowe sieci z ReLU i leaky ReLU podąża za wzrostem funkcji dłużej niż przedłużenie stałe sieci z funkcjami tanh i sigmoidalną.

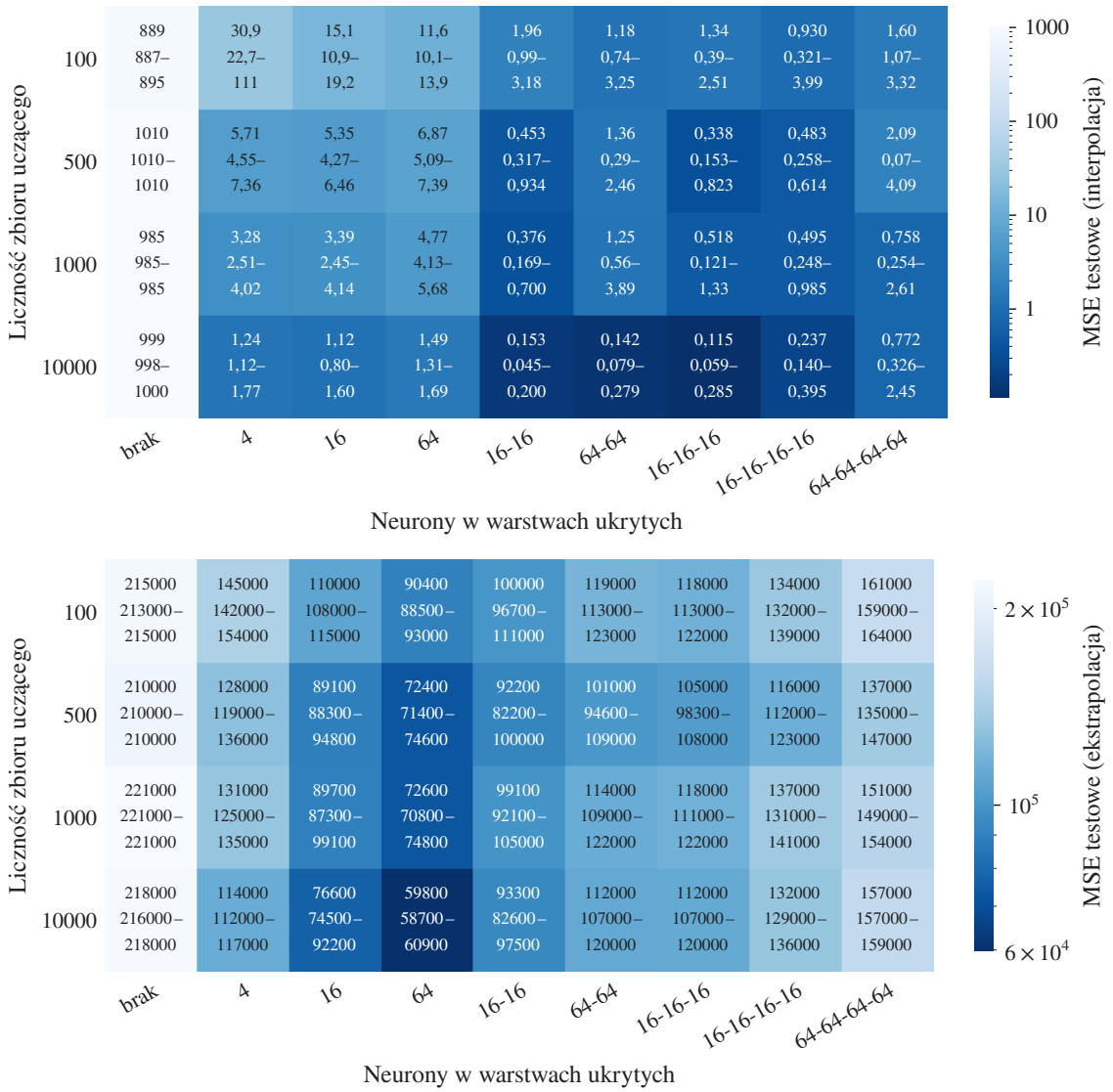
8.2 Architektura sieci

Wyniki badania architektur dla zbioru `activation` przedstawiono na Rysunku 24. Sieć bez warstw ukrytych, czyli model liniowy, ma w zakresie zbioru uczącego medianę MSE od 693 do 708. Sieci z warstwami ukrytymi osiągają od 0,002 25 do 0,470, czyli co najmniej o trzy rzędy wielkości mniej. Ich kolejność zmienia się z licznością, a kwartyle są szerokie, więc nie wskazują jednej najlepszej architektury. Poza zakresem sieci z jedną warstwą ukrytą mają medianę MSE od 4,55 do 54,4, a sieci z co najmniej dwiema warstwami od 0,492 do 10,3. Największe błędy wśród sieci z warstwami ukrytymi ma jedna warstwa z 64 neuronami, od 18,9 do 54,4.



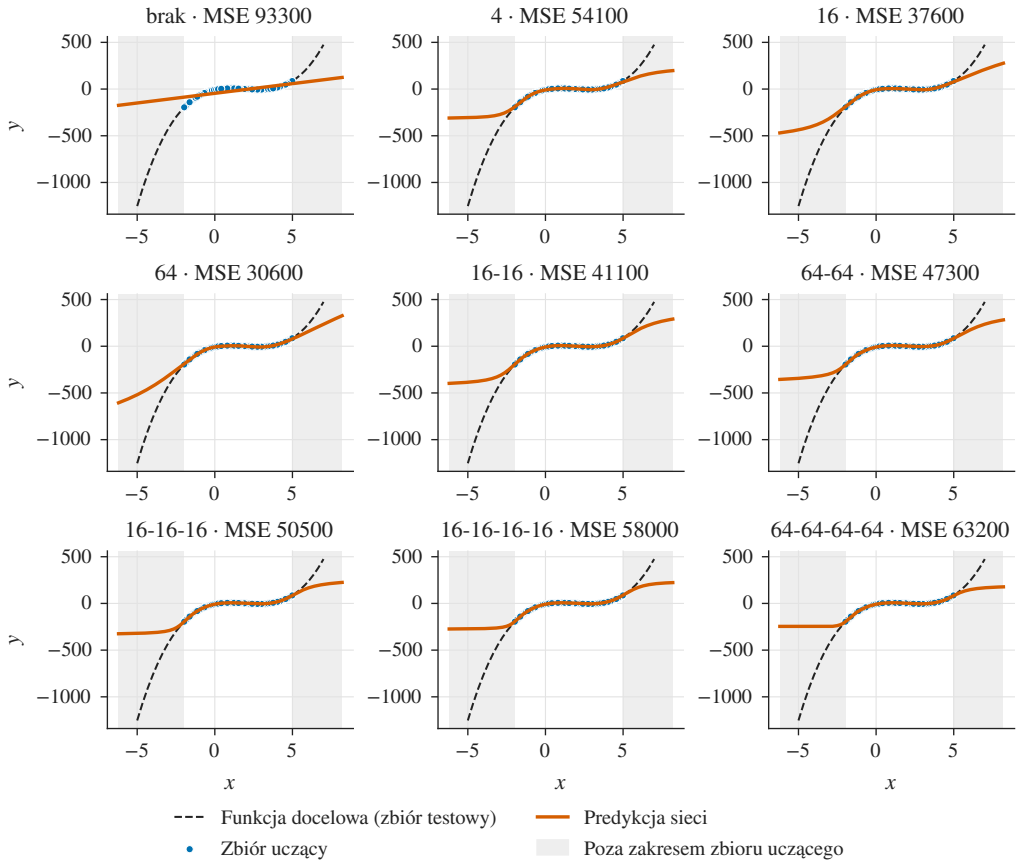
Rysunek 24: MSE testowe na zbiorze `activation` w zależności od architektury i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach. Górny wykres: w zakresie zbioru uczącego. Dolny wykres: poza zakresem zbioru uczącego.

Wyniki dla zbioru `cube` przedstawiono na Rysunku 25. Model liniowy ma w zakresie zbioru uczącego medianę MSE od 889 do 1010. Przy każdej liczności każda sieć z co najmniej dwiema warstwami ukrytymi jest lepsza od każdej sieci z jedną warstwą. Przy liczności 100 mediany wynoszą od 0,930 do 1,96 wobec od 11,6 do 30,9. Poza zakresem najmniejszy błąd ma jedna warstwa z 64 neuronami (od 59 800 do 90 400), a największy spośród sieci z warstwami ukrytymi sieć 64-64-64-64 (od 137 000 do 161 000).



Rysunek 25: MSE testowe na zbiorze `cube` w zależności od architektury i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach. Górny wykres: w zakresie zbioru uczącego. Dolny wykres: poza zakresem zbioru uczącego.

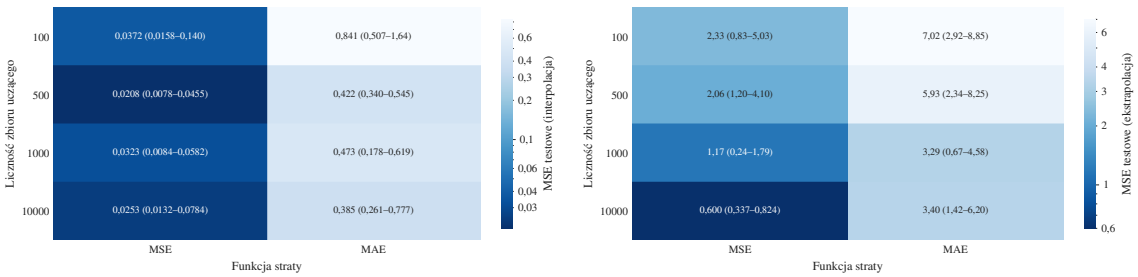
Funkcje sieci o różnych architekturach przy liczności 1000 pokazano na Rysunku 26, w tej samej postaci co na Rysunku 22. Model liniowy nie dopasowuje się do funkcji nawet w zakresie danych uczących. Pozostałe sieci pokrywają się z funkcją docelową w zakresie danych, a poza nim wypłaszczają się, podczas gdy funkcja docelowa rośnie dalej. Sieci głębokie wypłaszczają się najszybciej, a sieć z jedną warstwą 64 neuronów przedłuża wzrost najdłużej, stąd jej najmniejszy błąd poza zakresem.



Rysunek 26: Funkcja sieci na zbiorze `cube` przy liczności 1000 dla każdej architektury: uczenie o MSE testowym najbliższym medianie po ziarnach i jego MSE testowe. Szare pasy oznaczają obszar poza zakresem zbioru uczącego.

8.3 Funkcje straty

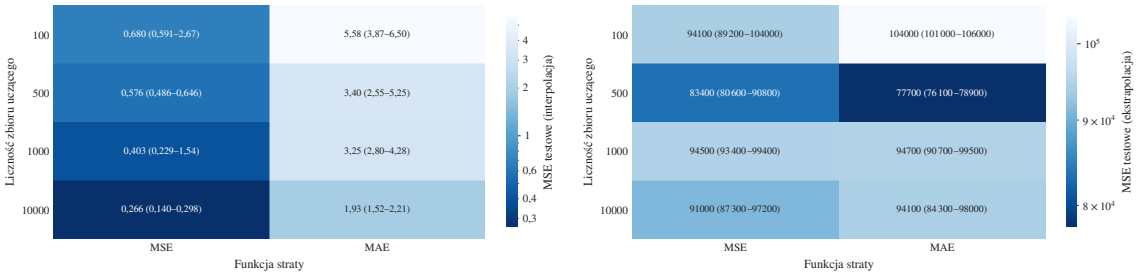
Wyniki badania funkcji straty przedstawiono na Rysunkach 27 i 28. Sieć uczona na MSE jest w zakresie zbioru uczącego wyraźnie lepsza od sieci uczonej na MAE. Mediana MSE sieci uczonej na MAE jest od 14,6 do 22,6 razy większa w zbiorze `activation` i od 5,91 do 8,21 razy większa w zbiorze `cube`. Poza zakresem sieć uczona na MSE jest lepsza w zbiorze `activation` (od 2,81 do 5,66 razy), a w zbiorze `cube` stosunek median wynosi od 0,93 do 1,10, więc nie ma systematycznej różnicy.



(a) w zakresie zbioru uczącego

(b) poza zakresem zbioru uczącego

Rysunek 27: MSE testowe na zbiorze `activation` w zależności od funkcji straty i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach.



(a) w zakresie zbioru uczącego (b) poza zakresem zbioru uczącego

Rysunek 28: MSE testowe na zbiorze `cube` w zależności od funkcji straty i liczności zbioru uczącego: mediana i kwartyle po 10 ziarnach.

Miara MSE z definicji sprzyja sieci, która ją minimalizuje. Dlatego porównano też średni błąd bezwzględny na całym zbiorze testowym. W zbiorze `activation` sieć uczona na MSE jest lepsza także w tej mierze, z medianą od 0,374 do 0,619 wobec od 0,763 do 1,27 dla sieci uczonej na MAE. W zbiorze `cube` błąd bezwzględny na całym zbiorze testowym zależy głównie od punktów poza zakresem, a mediany obu sieci różnią się o co najwyżej 8%.

Sieć uczona na MAE ma również większe MSE na zbiorze uczącym, ponad 14 razy w zbiorze `activation` i ponad 7 razy w zbiorze `cube`. Gorszy wynik w zakresie danych wynika więc z mniej dokładnego dopasowania do zbioru uczącego, a nie z gorszego uogólniania. Minimalizacja MAE prowadzi do mediany warunkowej zamiast średniej [3, rozdz. 1.5.5], więc jest mniej wrażliwa na pojedyncze duże błędy, np. punkty odstające. W badanych zbiorach punkty uczące leżą blisko funkcji docelowej (podrozdział 2.2), więc ta zaleta nie ma okazji się ujawnić.

9 Analiza przekrojowa

W rozdziale zestawiono zależności wspólne dla wszystkich badań: wpływ liczności zbioru uczącego na przeuczenie i ekstrapolację oraz rozrzut wyników między ziarnami generatora, który uzasadnia uczenie każdego wariantu wielokrotnie.

9.1 Rozmiar zbioru uczącego i przeuczenie

Dokładność na zbiorze uczącym i testowym dla wybranych architektur zestawiono w Tabeli 12. W obu zbiorach różnica między nimi maleje wraz z licznością. W zbiorze `three_gauss` przy liczności 100 wynosi od 0,53 punktu procentowego dla sieci bez warstw ukrytych do 8,19 dla sieci 64-64, a przy liczności 10 000 nie przekracza 0,24 dla żadnego wariantu. W zbiorze `simple` różnica wynosi przy liczności 100 od 0,61 do 1,06 punktu procentowego, a przy liczności 10 000 co najwyżej 0,08.

Tabela 12: Dokładność na zbiorze uczącym i testowym w zależności od liczności zbioru uczącego: średnia po 10 ziarnach, w procentach.

Zbiór	Warstwy ukryte	Zbiór oceny	100	500	1000	10 000
`simple`	16-16	uczący	100,00	99,70	99,46	99,44
		testowy	98,97	99,08	99,38	99,40
`three_gauss`	brak	uczący	92,77	92,87	92,33	92,65
		testowy	92,24	92,60	92,55	92,55
	16-16	uczący	96,57	94,85	93,71	93,79
		testowy	92,72	93,29	93,51	93,64
	64-64-64-64	uczący	98,47	96,90	94,04	93,68
		testowy	91,28	92,66	92,87	93,45

Im więcej parametrów ma sieć w stosunku do liczby próbek uczących, tym większa różnica. Widać to w Tabeli 12 na przykładzie sieci 64-64-64-64, która ma 12 867 parametrów (Tabela 9) przy 300 próbkach uczących. Z tego powodu architektura ma największe znaczenie dla małych zbiorów. Rozstęp średnich dokładności sieci z warstwami ukrytymi na zbiorze `three_gauss` wynosi 1,86 punktu procentowego przy liczności 100 i 0,26 przy liczności 10 000.

W regresji przeuczenie w tym sensie nie występuje. Mediany MSE na zbiorze uczącym i w zakresie zbioru uczącego na zbiorze testowym są prawie równe dla wszystkich wariantów, a ich stosunek wynosi od 0,78 do 1,14. Przykłady zestawiono w Tabeli 13. Punkty uczące leżą blisko funkcji docelowej (podrozdział 2.2), więc sieć, która dobrze dopasowuje się do nich, dobrze przybliża też tę funkcję w tym samym przedziale.

Tabela 13: MSE na zbiorze uczącym oraz na zbiorze testowym w zakresie i poza zakresem zbioru uczącego w zależności od liczności zbioru uczącego: mediana po 10 ziarnach. Sieci 32-32 mają funkcję aktywacji tanh.

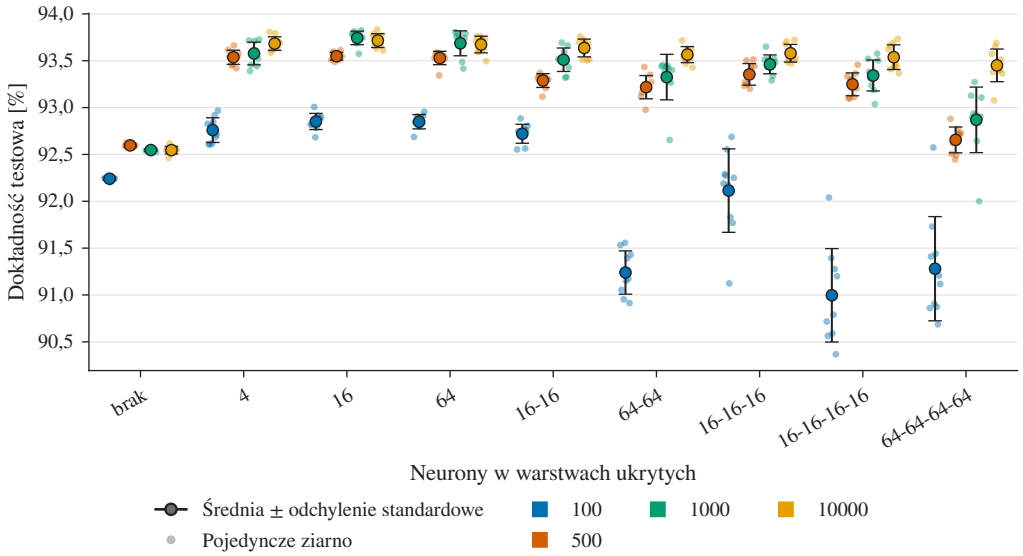
Zbiór	Warstwy ukryte	MSE	100	500	1000	10 000
`activation`	32-32	uczące	0,0407	0,0209	0,0324	0,0257
		w zakresie	0,0372	0,0208	0,0323	0,0253
		poza zakresem	2,33	2,06	1,17	0,600
	64-64-64-64	uczące	0,104	0,482	0,208	0,0531
		w zakresie	0,104	0,470	0,207	0,0527
		poza zakresem	1,07	1,41	1,59	1,05
`cube`	32-32	uczące	0,836	0,601	0,380	0,272
		w zakresie	0,680	0,576	0,403	0,266
		poza zakresem	94 100	83 400	94 500	91 000
	64-64-64-64	uczące	1,43	2,37	0,678	0,772
		w zakresie	1,60	2,09	0,758	0,772
		poza zakresem	161 000	137 000	151 000	157 000

Większy zbiór uczący poprawia dopasowanie w zakresie danych znacznie bardziej niż poza nim.

Dla każdej sieci z warstwami ukrytymi obliczono, ile razy mediana MSE przy liczności 100 jest większa niż przy liczności 10 000. W zakresie zbioru uczącego mediana tego stosunku po sieciach wynosi 4,90 w zbiorze `activation` i 7,81 w zbiorze `cube`. Poza zakresem te stosunki wynoszą 2,86 i 1,09. W zbiorze `cube` MSE poza zakresem praktycznie nie zależy od liczności. Dodatkowe punkty z tego samego przedziału nie niosą informacji o przebiegu funkcji poza nim, więc o błędzie ekstrapolacji decyduje kształt przedłużenia funkcji sieci (podrozdział 8.1). We wszystkich tych porównaniach efekt liczności zawiera też większą liczbę iteracji uczenia (Tabela 11).

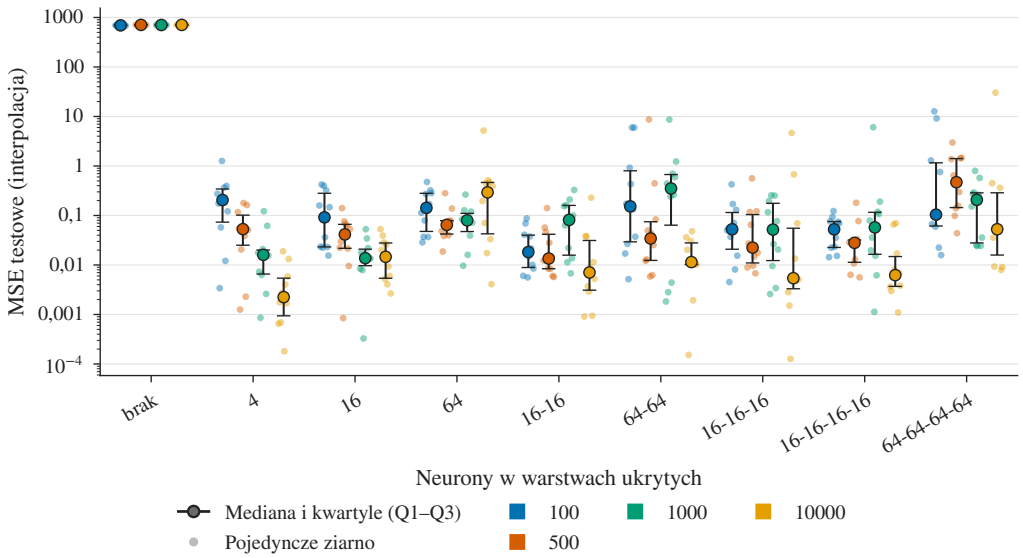
9.2 Powtarzalność między ziarnami

Rozrzut dokładności między ziarnami przedstawiono na przykładzie badania architektur na zbiorze `three_gauss` na Rysunku 29. We wszystkich badaniach klasyfikacji odchylenie standardowe dokładności wynosi od 0,01 do 0,56 punktu procentowego, z medianą 0,12. Największe jest dla dużych sieci przy liczności 100 na zbiorze `three_gauss` (od 0,45 do 0,56) i przy liczności 1000 na zbiorze `simple` (od 0,38 do 0,49). Sieć bez warstw ukrytych na zbiorze `three_gauss` daje prawie ten sam wynik dla każdego ziarna, z odchyleniem od 0,010 do 0,040 punktu procentowego. Entropia krzyżowa modelu regresji logistycznej jest wypukłą funkcją jego parametrów, więc nie ma minimów lokalnych gorszych od globalnego [3, rozdz. 4.3.3], więc każde ziarno prowadzi do niemal tego samego rozwiązania.



Rysunek 29: Dokładność testowa na zbiorze `three_gauss` dla każdej architektury, liczności zbioru uczącego i ziarna: małe punkty to pojedyncze ziarna, duże punkty z wąsami to średnia i odchylenie standardowe po 10 ziarnach.

W regresji rozrzut jest znacznie większy, co pokazuje Rysunek 30 dla badania architektur na zbiorze `activation`. We wszystkich badaniach regresji dla sieci z warstwami ukrytymi stosunek trzeciego kwartyla MSE w zakresie zbioru uczącego do pierwszego wynosi od 1,28 do 60,2, z medianą 4,28, i w 82% wariantów jest nie mniejszy niż 2. W 30% wariantów najgorsze ziarno ma MSE co najmniej 10 razy większe od mediany, a w skrajnym przypadku 858 razy większe (sieć 16-16-16 na zbiorze `activation`, liczność 10 000).



Rysunek 30: MSE testowe w zakresie zbioru uczącego na zbiorze `activation` dla każdej architektury, liczności zbioru uczącego i ziarna: małe punkty to pojedyncze ziarna, duże punkty z wąsami to mediana i kwartyle po 10 ziarnach.

Skutki takiego rozrzutu dla porównań sprawdzono, wybierając najlepszy wariant na podstawie wyników tylko jednego ziarna. Dla każdego badania, liczności i ziarna porównano wariant najlepszy dla tego ziarna z wariantem najlepszym według średniej dokładności albo mediany MSE w zakresie z 10 ziaren. W klasyfikacji wybór różnił się w 92 z 240 przypadków (38%), ale średnia dokładność wybranego wariantu była niższa od najlepszej w połowie z nich o nie więcej niż 0,046 punktu procentowego, a najwyżej o 0,356. Różnice między najlepszymi wariantami są w klasyfikacji tak małe, że błędny wybór niewiele kosztuje. W regresji wybór różnił się w 94 z 240 przypadków (39%). Mediana MSE wybranego wariantu była wtedy w połowie z nich co najmniej 2,07 razy większa od najlepszej, w 6 przypadkach co najmniej 10 razy, a najwyżej 131 razy. Pojedyncze uczenie nie wystarcza więc do porównania wariantów, zwłaszcza w regresji, co uzasadnia uczenie każdego wariantu dla 10 ziaren i podawanie mediany i kwartyli.

10 Podsumowanie i wnioski

Zaimplementowano od podstaw perceptron wielowarstwowy i zbadano na czterech zbiorach wpływ funkcji aktywacji, architektury i funkcji straty, ucząc każdy wariant dla 10 ziaren. W rozdziale zebrano najważniejsze wnioski oraz ograniczenia przeprowadzonych badań.

10.1 Wnioski

Za mocne uznano efekty wyraźnie większe od rozrzutu wyników między ziarnami, a pozostałe opisano jako słabe (podrozdział 6.3).

Klasyfikacja.

- Sieć bez warstw ukrytych wystarcza dla zbioru `simple`, w którym klasy rozdziela prosta. Przy licznościach 100 i 10 000 jest najlepsza i ma najmniejszy rozrzut między ziarnami (podrozdział 7.2).
- W zbiorze `three_gauss` sieć bez warstw ukrytych traci około punktu procentowego, bo nie wyraża krzywych granic. Już jedna warstwa z 4 neuronami usuwa tę różnicę (efekt mocny). Żaden wariant nie przekracza średnio 93,74%, co jest zgodne z nakładaniem się klas.
- Duże sieci przeuczają się przy małym zbiorze uczącym. Sieci 64-64, 16-16-16-16 i 64-64-64-64 przy liczności 100 na klasę osiągają na zbiorze testowym od 91,00% do 91,28%, wobec 92,85% dla jednej warstwy z 16 neuronami (efekt mocny).
- Funkcje aktywacji i funkcje straty różnią się słabo. Różnice między funkcjami straty nie przekraczają 0,09 punktu procentowego, a między funkcjami aktywacji wynoszą co najwyżej kilka dziesiątych punktu procentowego i ujawniają się głównie przy najmniejszych zbiorach (podrozdziały 7.1 i 7.3).

Regresja.

- O błędzie poza zakresem zbioru uczącego decyduje kształt przedłużenia funkcji sieci. Sieć z funkcją tanh nasyca się tak jak funkcja docelowa zbioru `activation` i ma tam medianę MSE od 0,600 do 2,33, a sieci z ReLU i leaky ReLU przedłużają funkcję prostą i mają od 1410 do 2200 (efekt mocny, podrozdział 8.1).
- Żadna z badanych sieci nie przedłuża funkcji zbioru `cube`, która na krańcach rośnie szybciej niż prosta. Najmniejszy błąd poza zakresem mają sieci, których przedłużenie najdłużej podąża za wzrostem: z ReLU i leaky ReLU oraz z jedną szeroką warstwą.
- W zakresie zbioru uczącego zbioru `cube` każda sieć z co najmniej dwiema warstwami ukrytymi jest dokładniejsza od każdej sieci z jedną warstwą, przy każdej liczności (podrozdział 8.2).
- Uczenie na MSE daje w zakresie zbioru uczącego od 5,91 do 22,6 razy mniejsze MSE niż uczenie na MAE. W zbiorze `activation` sieć uczona na MSE jest lepsza także w mierze MAE (podrozdział 8.3).

Liczność zbioru i powtarzalność.

- Różnica między dokładnością na zbiorze uczącym i testowym maleje z licznością. W zbiorze `three_gauss` wynosi do 8,19 punktu procentowego przy liczności 100 i nie przekracza 0,24 przy 10 000, a wraz z nią maleje znaczenie wyboru architektury (podrozdział 9.1).
- Większy zbiór uczący poprawia regresję w zakresie danych znacznie bardziej niż poza nim. W zbiorze `cube` MSE poza zakresem praktycznie nie zależy od liczności.
- Rozrzut wyników między ziarnami jest w klasyfikacji niewielki, a w regresji duży. Wybór najlepszego wariantu na podstawie jednego ziarna różni się od wyboru na podstawie 10 ziaren w 38% przypadków w klasyfikacji i 39% w regresji. W regresji taki wybór może wskazać wariant o nawet 131 razy większej medianie MSE (podrozdział 9.2). Porównania wariantów wymagają więc wielu ziaren i statystyk odpornych na pojedyncze złe wyniki.

10.2 Ograniczenia i dalsze prace

Wnioski obowiązują w warunkach, w których przeprowadzono badania:

- każde badanie zmieniało jeden czynnik względem konfiguracji bazowej, więc nie wiadomo, czy czynniki na siebie oddziałują, np. czy inna funkcja aktywacji wymaga innej architektury,
- wszystkie warianty uczono ze stałym budżetem iteracji i jednym współczynnikiem uczenia, bez zbioru walidacyjnego i wczesnego zatrzymania. Dokładność testowa dużych sieci przy małych zbiorach osiągała w trakcie uczenia maksimum wyraźnie wyższe od wartości końcowej (Rysunek 18), więc z wczesnym zatrzymaniem lub innym współczynnikiem uczenia mogłyby one wypaść lepiej,
- zbiory są proste: wejścia klasyfikacji mają dwa wymiary i są tylko dwie lub trzy klasy, a punkty uczące regresji leżą blisko funkcji docelowej. Część efektów, np. przewaga entropii krzyżowej nad MSE albo odporność MAE na punkty odstające, może ujawnić się dopiero na trudniejszych danych.

Część druga projektu dotyczy rozpoznawania cyfr ze zbioru MNIST, czyli klasyfikacji wektorów o wielu wymiarach do dziesięciu klas. Wyniki części pierwszej wskazują, że będą tam potrzebne zbiór walidacyjny, wczesne zatrzymanie i dobór współczynnika uczenia, a porównania wariantów powinny nadal opierać się na wielu ziarnach.

Bibliografia

[1] Ian Goodfellow, Yoshua Bengio i Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. URL: https://www.deeplearningbook.org.

[2] David E. Rumelhart, Geoffrey E. Hinton i Ronald J. Williams. „Learning representations by back-propagating errors". W: *Nature* 323.6088 (1986), s. 533–536. DOI: 10.1038/323533a0.

[3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[4] Ilya Sutskever, James Martens, George Dahl i Geoffrey Hinton. „On the importance of initialization and momentum in deep learning". W: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. T. 28. Proceedings of Machine Learning Research. 2013, s. 1139–1147.

[5] Diederik P. Kingma i Jimmy Ba. „Adam: A Method for Stochastic Optimization". W: *3rd International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6980.

[6] Xavier Glorot i Yoshua Bengio. „Understanding the difficulty of training deep feedforward neural networks". W: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. T. 9. Proceedings of Machine Learning Research. 2010, s. 249–256.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren i Jian Sun. „Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". W: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015, s. 1026–1034.

[8] The Rand Project Developers. *rand: Random number generators and other randomness functionality*. Wersja 0.10.3. URL: https://crates.io/crates/rand.

[9] Daniel J. Bernstein. „ChaCha, a variant of Salsa20". W: *Workshop Record of SASC 2008: The State of the Art of Stream Ciphers*. 2008.

[10] Keyulu Xu i in. „How Neural Networks Extrapolate: From Feedforward to Graph Neural Networks". W: *9th International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2009.11848.

A Odtworzenie wyników

Do odtworzenia wyników potrzebne są kompilator języka Rust w wersji 1.88.0, wskazanej w pliku `rust-toolchain.toml`, oraz Python w wersji co najmniej 3.13 z menedżerem pakietów `uv`, który instaluje zależności w wersjach zapisanych w pliku `uv.lock`. Ścieżki w plikach konfiguracyjnych są względne, dlatego polecenia uruchamia się z katalogu głównego repozytorium. Polecenia przedstawiono na Listingu A.1.

Listing A.1 Polecenia odtwarzające wyniki, uruchamiane z katalogu głównego repozytorium.

```
# zbudowanie programu i instalacja zależności Pythona
cargo build --release --manifest-path src/compute/Cargo.toml
uv sync --project src/analysis

# eksperyment pokazowy: uczenie i wykresy
cargo run --release --manifest-path src/compute/Cargo.toml -- \
    train data/experiments/classification/showcase-three-gauss
uv run --project src/analysis src/analysis/main.py \
    plot data/experiments/classification/showcase-three-gauss

# jedno badanie: eksperymenty, uczenie, zestawienie wyników
cargo run --release --manifest-path src/compute/Cargo.toml -- \
    generate data/experiments/classification/simple-activations
cargo run --release --manifest-path src/compute/Cargo.toml -- \
    train data/experiments/classification/simple-activations
uv run --project src/analysis src/analysis/main.py \
    aggregate data/experiments/classification/simple-activations --plot-runs

# wszystkie badania części pierwszej (PowerShell 7)
pwsh -File scripts/run-studies.ps1
```

Konfiguracje i wyniki znajdują się w następujących miejscach:

- konfiguracje eksperymentów pokazowych: `data/experiments/<zadanie>/showcase-<zbiór>/config.toml`,
- konfiguracje badań: `data/experiments/<zadanie>/<zbiór>-<czynnik>/study.toml`,
- wykresy eksperymentów pokazowych: katalog `plots/` eksperymentu,
- zestawienia badań: katalog `results/` badania.

Katalog `<zadanie>` to `classification` albo `regression`. Eksperymenty pokazowe używają ziarna 1, a badania ziaren od 1 do 10.